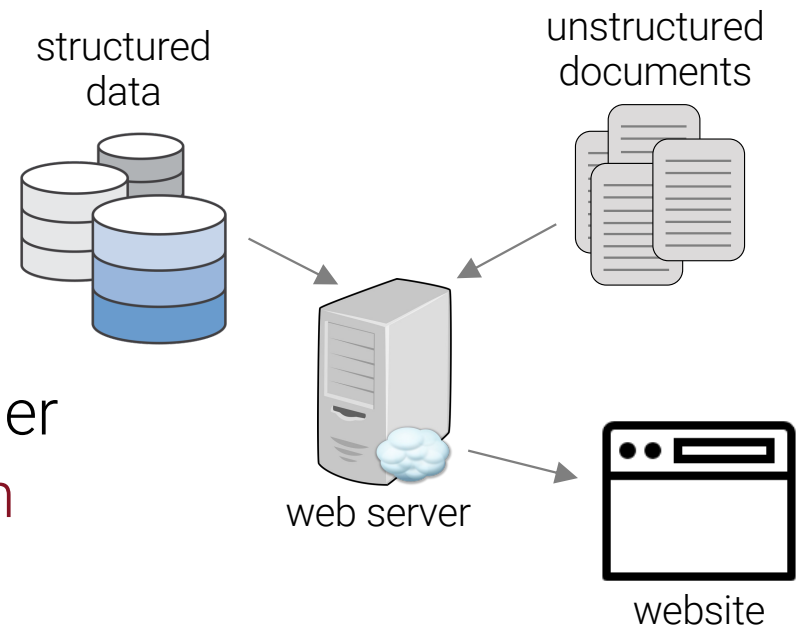


Module 2: Structured data extraction

Structured data extraction

- Web crawlers use parsers to extract content from web pages. Content further processed using **information extraction techniques**.
- **Web extraction:**
 - Extraction of automatically generated content (**wrapper-based extraction**).
 - Extraction from more natural, unstructured, human-written text (**natural language processing**).



DELL AIO računalnik Inspiron 7777 i7-8700T/16GB/SSD256GB+1TB/GTX1050/27FHD/W10P (5397184169834)

Znamka: [DELL](#) Naša številka: 1257134



Brezplačna dostava

DELL AIO računalnik Inspiron 7777 i7-8700T/16GB/SSD256GB+1TB/GTX1050/27FHD/W10P (5397184169834)

Računalnik vse v enem (AIO) Inspiron 7777 vsebuje vse kar potrebujete, zaslon velikosti 68,58 cm (27"), miško in tipkovnico. Pogarja ga izjemno zmogljiv procesor Intel Core i7-8700T, ki mu pomaga 16 GB DDR4 delovnega pomnilnika. Za visoko grafično zmogljivost pa bo poskrbela grafična kartica NVIDIA GeForce GTX 1050. Hitro in brezhibno delovanje zagotavlja SSD disk kapacitete 256 GB v kombinaciji z 1 TB trdim diskom, ki ponuja dodatno shrambo za datoteke. Nameščen je operacijski sistem Windows 10 Pro. [Celoten opis](#)

1.948,60 € vključno z DDV

Redna cena: 2.439,90 €, Prihranite 191,30 € (8 %)

Zadnji kos!

[Na zalogi – predvidena odprema: 11.12.](#)

[DODAJ V KOŠARICO](#)

[SLEDI CENI](#)

[PRIMERJAJ](#)

PREDSTAVITEV

TEHNIČNE PODROBNOSTI

VEČ O ARTIKLU

TEHNIČNE PODROBNOSTI

Operacijski sistem	Windows 10 Pro
Procesor	Intel Core i7
Frekvenca	2400 MHz
Spomin (RAM)	16 GB
Vrsta delovnega pomnilnika	DDR IV
Vrsta spominskega medija	HDD + SSD

DELL AIO računalnik Inspiron 7777 i7-8700T/16GB/SSD256GB+1TB/GTX1050/27FHD/W10P (5397184169834)

Znamka: [DELL](#) Naša številka: 1257134

Brezplačna dostava

DELL AIO računalnik Inspiron 7777 i7-8700T/16GB/SSD256GB+1TB/GTX1050/27FHD/W10P (5397184169834)

Računalnik vse v enem (AIO) Inspiron 7777 vsebuje vse kar potrebujete, zaslon velikosti 68,58 cm (27"), miško in tipkovnico. Poganja ga izjemno zmogljiv procesor Intel Core i7-8700T, ki mu pomaga 16 GB DDR4 delovnega pomnilnika. Za visoko grafično zmogljivost pa bo poskrbela grafična kartica NVIDIA GeForce GTX 1050. Hitro in brezhibno delovanje zagotavlja SSD disk kapacitete 256 GB v kombinaciji z 1 TB trdim diskom, ki ponuja dodatno shrambo za datoteke. Nameščen je operacijski sistem Windows 10 Pro. [Celoten opis](#)

▼<p class="mb-25 pro-description--short pro-hide-mobile" itemprop="description">
"Računalnik vse v enem (AIO) Inspiron 7777 vsebuje vse kar potrebujete, zaslon velikosti 68,58 cm (27"), miško in tipkovnico. Poganja ga izjemno zmogljiv procesor Intel Core i7-8700T, ki mu pomaga 16 GB DDR4 delovnega pomnilnika. Za visoko grafično zmogljivost pa bo poskrbela grafična kartica NVIDIA GeForce GTX 1050. Hitro in brezhibno delovanje zagotavlja SSD disk kapacitete 256 GB v kombinaciji z 1 TB trdim diskom, ki ponuja dodatno shrambo za datoteke. Nameščen je operacijski sistem Windows 10 Pro. "
Celoten opis
</p>



SLEDI CENI PRIMERJAJ

PREDSTAVITEV

TEHNIČNE PODROBNOSTI

VEČ O ARTIKLU

TEHNIČNE PODROBNOSTI

Operacijski sistem	Windows 10 Pro
Procesor	Intel Core i7
Frekvenca	2400 MHz
Spomin (RAM)	16 GB
Vrsta delovnega pomnilnika	DDR IV
Vrsta spominskega medija	HDD + SSD

DELL AIO računalnik Inspiron 7777 i7-8700T/16GB/SSD256GB+1TB/GTX1050/27FHD/W10P (5397184169834)

Znamka: [DELL](#) Naša številka: 1257134

Brezplačna dostava

DELL AIO računalnik Inspiron 7777 i7-8700T/16GB/SSD256GB+1TB/GTX1050/27FHD/W10P (5397184169834)

Računalnik vse v enem (AIO) Inspiron 7777 vsebuje vse kar potrebujete, zaslon velikosti 68,58 cm (27"), miško in tipkovnico. Poganja ga izjemno zmogljiv procesor Intel Core i7-8700T, ki mu pomaga 16 GB DDR4 delovnega pomnilnika. Za visoko grafično zmogljivost pa bo poskrbela grafična kartica NVIDIA GeForce GTX 1050. Hitro in brezhibno delovanje zagotavlja SSD disk kapacitete 256 GB v kombinaciji z 1 TB trdim diskom, ki ponuja dodatno shrambo za datoteke. Nameščen je operacijski sistem Windows 10 Pro. [Celoten opis](#)

1.948,60 € vključno z DDV

Redna cena: ~~2.139,90 €~~, Prihranite 191,30 € (8 %)

```
<div class="price-wrapper">
  <div class="pro-price-wrapper">
    <b class="pro-price con-emphasize font-primary--bold mr-5">1.948,60 €</b>
    "vključno z DDV"
  <span class="con-emphasize m-0 base-price">
    <span class="lay-inline">Redna cena: </span>
    <del>2.139,90 €</del>
    ",Prihranite 191,30 € "
    <span class="moc-percentage">(8&nbsp;%;</span>
  </span>
</div>
</div>
```

PREDSTAVITEV

TEHNIČNE PODROBNOSTI

Operacijski sistem	Windows 10 Pro
Procesor	Intel Core i7
Frekvenca	2400 MHz
Spomin (RAM)	16 GB
Vrsta delovnega pomnilnika	DDR IV
Vrsta spominskega medija	HDD + SSD

DELL AIO r
8700T/160
OP (53971)

Znamka: [DELL](#) Na



```
▼<div class="content-tabs">
  ▶<div id="product-detail-description" ng-show="tabDescription" class="ng-hide">...</div>
  ▼<div id="product-detail-parameters" ng-show="tabParameters" class>
    ▼<div class="panel-outer">
      ::before
      <h2 class="con-h1 content-tab-title">Tehnične podrobnosti</h2>
      ▼<section class="panel-inner">
        ::before
        <h3 class="lay-off">Parametri artikla %1% %2%</h3>
        ▼<table class="tbl tbl--condensed tbl--block mb-0">
          ▼<tbody>
            ▼<tr>
              <th class="con-normal">Operacijski sistem</th>
              <td class="con-bold con-right">Windows 10 Pro</td>
            </tr>
            ▼<tr>
              <th class="con-normal">Procesor</th>
              <td class="con-bold con-right">Intel Core i7</td>
            </tr>
            ▼<tr>
              <th class="con-normal">Frekvenca</th>
              ▼<td class="con-bold con-right">
                "2400
                "
                <abbr title="Megahertz">MHz</abbr>
              </td>
            </tr>
            ▶<tr>...</tr>
            ▶<tr>...</tr>
            ▶<tr>...</tr>
            ▶<tr>...</tr>
            ▶<tr>...</tr>
            ▶<tr>...</tr>
            ▶<tr>...</tr>
```

D/W10P

cm (27"),
16 GB DDR4
GeForce
1 TB trdim
Pro. [Celoten](#)

TEHNIČNE PODROBNOSTI

Extracted product data

» Računalništvo » Računalniki in strežniki » DELL AIO računalnik Inspiron 7777 i7-8700T/16GB/SSD256GB+1TB/GTX1050/27FHD/W10P (5397184169834)

DELL AIO računalnik Inspiron 7777 i7-8700T/16GB/SSD256GB+1TB/GTX1050/27FHD/W10P (5397184169834)

Znamka: DELL Naša številka: 1257134



Brezplačna dostava

DELL AIO računalnik Inspiron 7777 i7-8700T/16GB/SSD256GB+1TB/GTX1050/27FHD/W10P (5397184169834)
Računalnik vse v enem (AIO) Inspiron 7777 vsebuje vse kar potrebujete, zaslon velikosti 68,58 cm (27"), miško in tipkovnico. Pogonje ga izjemno zmogljiv procesor Intel Core i7-8700T, ki mu pomaga 16 GB DDR4 delovnega pomnilnika. Za visoko grafično zmogljivost pa bo poskrbela grafična kartica NVIDIA GeForce GTX 1050. Hitro in brezhibno delovanje zagotavlja SSD disk kapacitete 256 GB v kombinaciji z 1 TB trdim diskom, ki ponuja dodatno shrambo za datoteke. Nameščen je operacijski sistem Windows 10 Pro. [Celoten opis](#)

1.948,60 € vključno z DDV
Rokna cena: 2.148,00 €, Prihranilo 191,30 € (8 %)

Zadnji kos!

Na zalogi – predvidena odprema: 11.12.

[DODAJ V KOŠARICO](#)

[SLEDI CENI](#) [PRIMERJAJ](#)

PREDSTAVITEV

TEHNIČNE PODROBNOSTI

VEČ O ARTIKLU

TEHNIČNE PODROBNOSTI

Operacijski sistem	Windows 10 Pro
Procesor	Intel Core i7
Frekvenca	2400 MHz
Spomn (RAM)	16 GB
Vrsta delovnega pomnilnika	DDR 4
Vrsta spominskega medija	HDD + SSD
Kapaciteta trdega diska	1000 GB
Grafične kartice	NVIDIA GeForce GTX 1050
Spomn grafične kartice	4000 MB
Optični pogon	Nobena
Čitalnik kartic	da
LAN(RJ45)	da
USB 2.0	2 x

© Laboratory for Data Technologies, UL FRI

```
{
  "productName": "Inspiron 7777",
  "manufacturer": "Dell",
  "snippet": "Računalnik vsebuje zaslon velikosti 68,58 cm (27'), miško in tipkovnico. Hitro in brezhibno delovanje zagotavlja SSD disk ... Nameščen je Windows 10 Pro.",
  "priceVat": "1948.6",
  "priceNoVat": null,
  "priceUnit": "EUR",
  "characteristics": [
    {
      "attribute": "OS",
      "value": "Windows 10 Pro"
    }, {
      "attribute": "CPU",
      "value": "Intel Core i7"
    }, {
      "attribute": "RAM",
      "value": "16 GB"
    }, {
      "attribute": "GPU",
      "value": "NVIDIA GeForce GTX 1050"
    }, {
      "attribute": "GPU_RAM",
      "value": "4000 MB"
    }
  ]
}
```

Web data extraction techniques

- DOM,
- Regular expressions, XPath, XQuery
- String matching
- Tree matching, alignment
- DOM generation
- Wrappers

DOM - Document Object Model

- Tags – HTML backbone.
- Web extraction:
 - Parsing HTML to DOM
 - Extraction patterns represented as paths in DOM ...
 - Special languages, e.g. **XPATH** or **XQUERY**
- Further text processing may still be needed to identify proper part of a text node.
- Challenge: maintenance

Document Object Model

Za otroke

Ledeni zmaji prileteli na Kongresni...

Za starejše

Foto: Najlepša v vesolju je...

Prireditve.info

Grafike tam kjer sem bil, tam kjer...

RTV SLO

MMC

MMC ODŠTEVA

Rtvslo.si

Go

Novice

Šport

Kultura

Zabava

Ture avanture

TTX

RTV

Nogomet

Košarka

Rokomet

Odbojka

Hokej

Tenis

Atletika

Zimski športi

Zimski športi

POŠLJI

NATISNI

Obetavna treninga smuka Ilke Štuhec

Najboljši čas dneva je imela Ester Ledecka; tekma v torek ob 12.30

17. december 2018 ob 11:04

Val Gardena - MMC RTV SLO

Ilka Štuhec se je izvrstno odrezala na obeh ponedeljkovih treningih smuka v Val Gardeni. Na prvem je bila druga, na drugem pa šesta.

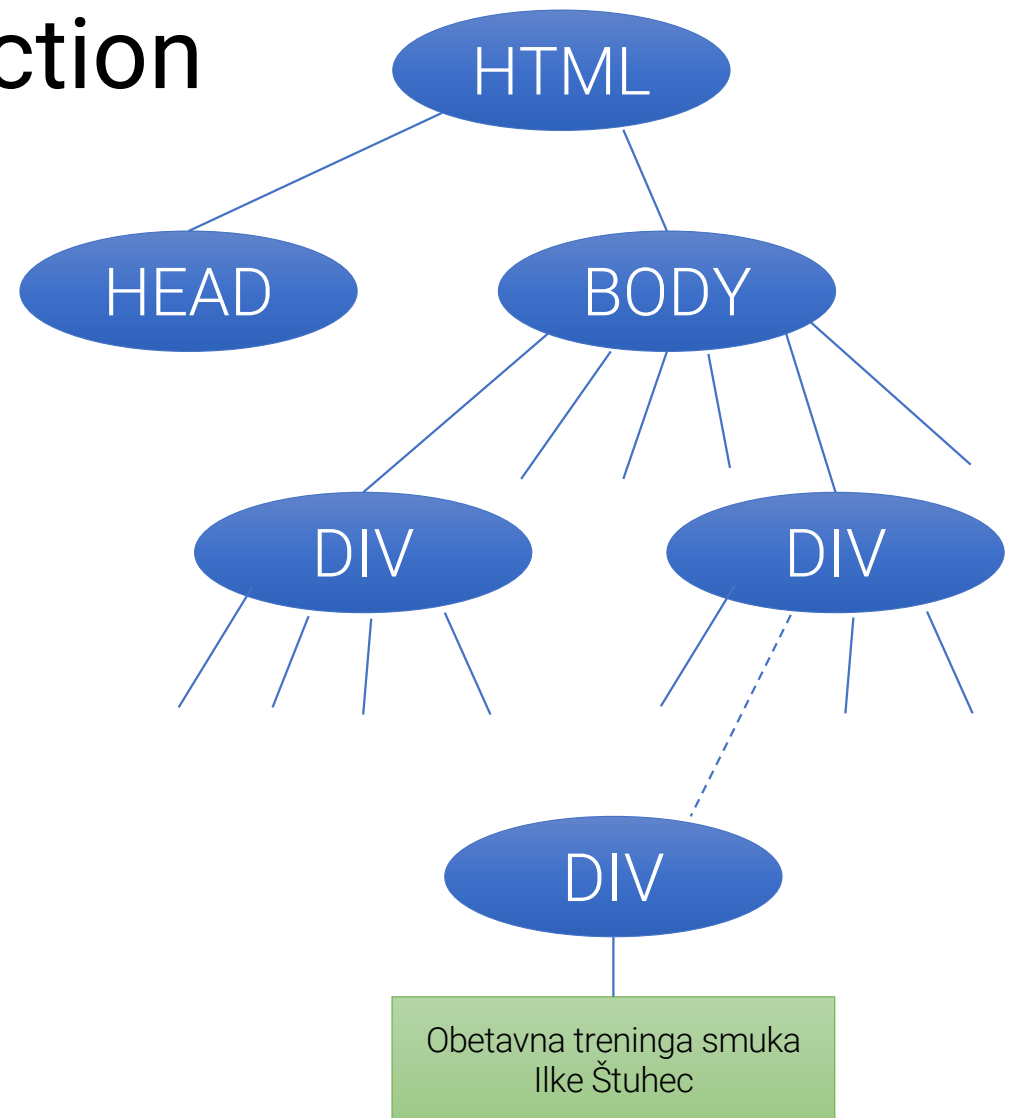
Štuhčeva se je na prvem treningu na progo pognala s številko ena in vodila vse do nastopa Ramone Siebenhofer, ki je imela številko 15. Avstrijka je bila hitrejša za dve stotinki. Tretja je bila Italijanka Nicol Delago (+0,37), četrta Kanadčanka Valerie Grenier

Ilka Štuhec bo v torek kandidirala za olimpijsko mesto. Foto: Reuters

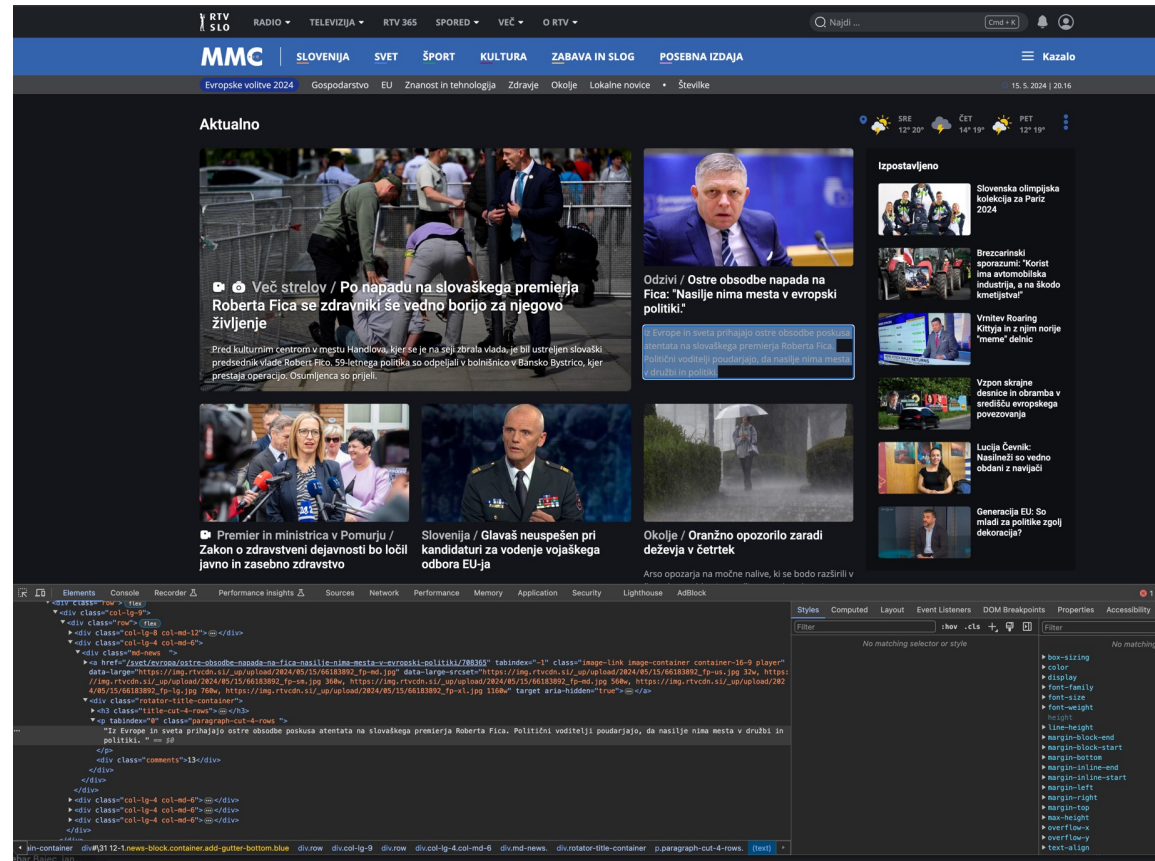
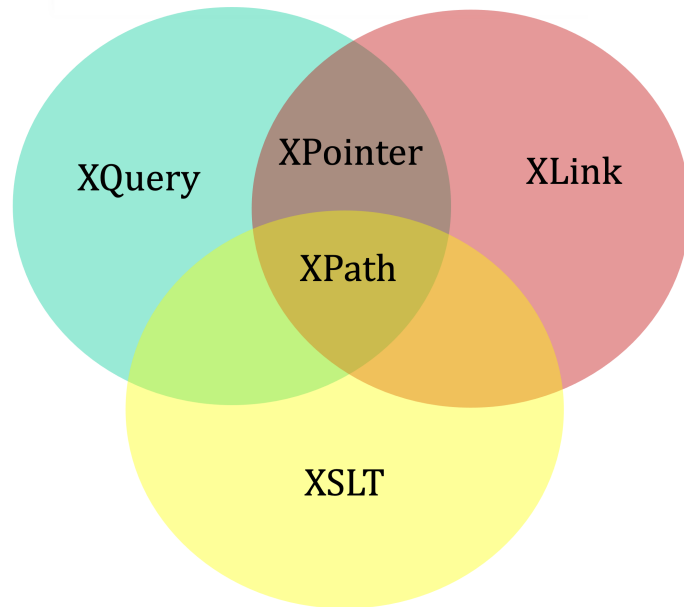
```
<html>
  <head>...</head>
  <body id="mp_s" class="submenu" style>
    <div id="cc-notification" class="cc-dark cc-bottom" style="display
    <style type="text/css">...</style>
    <div id="dashboard" class="realign">...</div>
    <script type="text/javascript">...</script>
    <style type="text/css">
      #site .head .search { margin-left: 23px; }
    </style>
    <div class="iAdserver adloaded" data-iadserver-zone="299" id="de1d
    <div id="site" class="realign">...</div>
    <div id="container" class="realign">
      <div class="bound" id="bound_block" style="padding-left: 7px;">
        <div id="contents">
          <script type="text/javascript">...</script>
          <style>...</style>
          <div class="cbox">
            <div class="header">...</div>
            <div class="contents">
              <div class="left">...</div>
              <div class="body" id="cnt_474962">
                <div id="newsbody">
                  <div id="newsblocks" class="fr" style="width: 302px; n
                  <h1>Obetavna treninga smuka Ilke Štuhec</h1> == $0
                  <div class="sub">Najboljši čas dneva je imela Ester Le
                  <div class="info">...</div>
                  <div class="lead">...</div>
```

Sample DOM Tree Extraction

```
<html>
  <head>...</head>
  <body id="mp_s" class="submenu" style>
    <div id="cc-notification" class="cc-dark cc-bottom" style="display
    <style type="text/css">...</style>
    <div id="dashboard" class="realign">...</div>
    <script type="text/javascript">...</script>
    <style type="text/css">
      #site .head .search { margin-left: 23px; }
    </style>
    <div class="iAdserver adloaded" data-iadserver-zone="299" id="deId
    <div id="site" class="realign">...</div>
    <div id="container" class="realign">
      <div class="bound" id="bound_block" style="padding-left: 7px;">
        <div id="contents">
          <script type="text/javascript">...</script>
          <style>...</style>
          <div class="cbox">
            <div class="header">...</div>
            <div class="contents">
              <div class="left">...</div>
              <div class="body" id="cnt_474962">
                <div id="newsbody">
                  <div id="newsblocks" class="fr" style="width: 302px; n
                  <h1>Obetavna treninga smuka Ilke Štuhec</h1> == $0
                  <div class="sub">Najboljši čas dneva je imela Ester Le
                  <div class="info">...</div>
                  <div class="lead">...</div>
                  <p>...</p>
```



XPath and XQuery



- `/html/body/div[13]/div[2]/div/div[1]/div/div[2]/div/div/p/text()`

Web page pre-processing

- Identification of text fields, anchor text, ...
 - Identification of main content blocks.
- Approaches:
- Visual cues-based partitioning
 - Tree matching
- HTML tags removal

The screenshot displays the AVTO.NET website interface. The top navigation bar includes the AVTO.NET logo, a search bar, and links for 'Objavi oglas', 'Parkirano', and 'moj.avto.net'. Below the navigation bar, a horizontal menu lists categories: 'Avto', 'Moto', 'Gospodarska', 'Mehanizacija', 'Prosti čas', and 'Deli in oprema'. The main content area is titled 'Rezultati: Osebna vozila Peugeot' and features a '+ Objavi nov oglas' button. On the left, a sidebar shows 'Prikazano 76 oglasov:' and filters for 'rubrika AVTO' (PRODAM, Peugeot, letnik (1.reg.) 2022), 'Omejite prikaz:', 'Storitve na daljavo' (Video klic / Virtual ogled, Online Nakup), 'Peugeot' (2008, 208, 3008, 308, 5008, 508, Rifter), 'Oblika' (limuzina, kombi / hatchback, karavan, enoprostorec / van, terensko vozilo / SUV), 'Starost' (rabljeno, testno), and 'Cena'. The main listing area shows three Peugeot models: Peugeot 3008 1.2 PureTech 130 EAT8 Allure Pack (2022, 890 km, 27.800 €), Peugeot 308 ACTIVE PACK 1.2 PURETECH 110 (2022, 39330 km, 19.890 €), and Peugeot 308 GT PACK SW 1.6 PHEV 225 EAT8 + POLNILNA POSTAJA (2022, 20200 km, 39.500 €). Each listing includes a car image, a table of specifications (1. registracija, Prevoženih, Gorivo, Menjalnik, Motor), and a price tag with a financing option.

String matching and tree matching

- String matching:
 - Edit distance (Levenshtein distance)
 - Jaro similarity
 - Jaro-Winkler similarity
- Tree matching:
 - Simple (weighted) tree matching
 - Multiple and partial alignment
 - ...

String edit distance

- Vladimir Levenshtein, 1965
- Minimum number of point mutations
 - character change
 - character insertion
 - character deletion
- Distance $d(s_1, s_2)$ of strings s_1 and s_2

String edit distance

- Distance $d(s_1, s_2)$ of strings s_1 and s_2

$$d(\epsilon, \epsilon) = 0$$

$$d(s, \epsilon) = d(\epsilon, s) = |s|$$

$$d(s_{1-} + c_1, s_{2-} + c_2) = \min \begin{cases} d(s_{1-}, s_{2-}) + p(c_1, c_2) \\ d(s_{1-} + c_1, s_{2-}) + 1 \\ d(s_{1-}, s_{2-} + c_2) + 1 \end{cases}$$

String edit distance - example

	#	w	a	b	t	i	s	e
#								
w								
e								
b								
s								
i								
t								
e								

String edit distance

- Time complexity – $O(|s_1||s_2|)$

- Normalized distance

$$ND(s_1, s_2) = \frac{d(s_1, s_2)}{(|s_1| + |s_2|)/2}$$

- Similarity: $1 - \text{distance}$

Jaro similarity

- Matthew A. Jaro, 1989
- Similarity *sim_j* of strings s_1 and s_2

$$sim_j = \begin{cases} 0 & \text{if } m = 0 \\ \frac{1}{3} \left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m} \right) & \text{otherwise} \end{cases}$$

- *m* – matching characters if not farther than $\left\lfloor \frac{\max(|s_1|, |s_2|)}{2} \right\rfloor - 1$
- *t* – half the number of transpositions not farther than $\left\lfloor \frac{\max(|s_1|, |s_2|)}{2} \right\rfloor - 1$

Jaro similarity - example

$$sim_j = \begin{cases} 0 & \text{if } m = 0 \\ \frac{1}{3} \left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m} \right) & \text{otherwise} \end{cases}$$

- m – matching characters if not farther than $\left\lfloor \frac{\max(|s_1|, |s_2|)}{2} \right\rfloor - 1$
- t – half the number of transpositions not farther than $\left\lfloor \frac{\max(|s_1|, |s_2|)}{2} \right\rfloor - 1$

```
Jaro-sim("website", "wabtise")
```

Jaro-Winkler similarity

- William E. Winkler, 1990
- Variant of Jaro, favorable to strings with matching prefix l
- Similarity sim_w of strings s_1 and s_2

$$sim_w = sim_j + l \cdot p \cdot (1 - sim_j)$$

- l – length of common prefix up to 4 characters
- p – scaling factor defined as 0.1

Jaro-Winkler similarity - example

$$sim_w = sim_j + l \cdot p \cdot (1 - sim_j)$$

- l – length of common prefix up to 4 characters
- p – scaling factor defined as 0.1

```
Jaro-Winkler-sim("website", "wabtise")
```

Tree edit distance matching algorithms

- A variation of classic string edit distance problem
- Let **A** and **B** be two ordered rooted trees. The problem is to find a sequence of operations to transform **A** into **B** (or the vice versa).
- Minimize the (weighted) cost of operations:
 - Node deletion
 - Node insertion
 - Node replacement

Tree edit distance matching algorithms

- Let $A[i_x]$ indicate the x -th node of the tree A in a preorder visit.
- A mapping M between A and B is a set of ordered pairs (i, j) , $i \in A, j \in B$, satisfying $\forall (i_1, j_1), (i_2, j_2) \in M$:
 - $i_1 = i_2$ if and only if $j_1 = j_2$
 - $A[i_1]$ is on the left of $A[i_2]$ if and only if $B[j_1]$ is on the left of $B[j_2]$
 - $A[i_1]$ is an ancestor of $A[i_2]$ if and only if $B[j_1]$ is an ancestor of $B[j_2]$

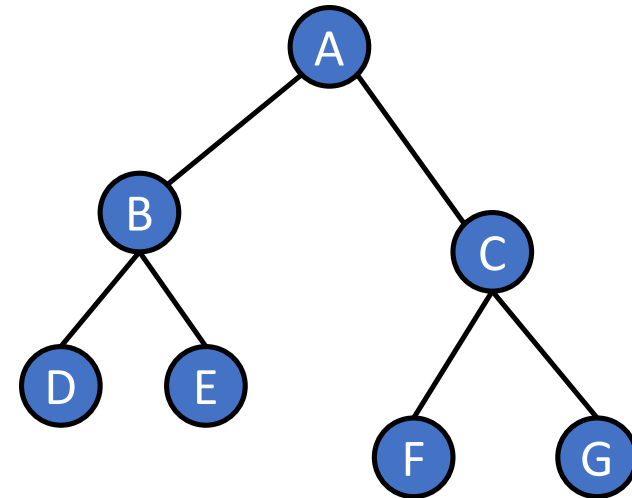
Tree visiting

- Inorder (Left, Root, Right)
- Preorder (Root, Left, Right)
- Postorder (Left, Right, Root)

Inorder: _____

Preorder: _____

Postorder: _____



The simple tree matching algorithm

- A computationally efficient solution for tree edit distance matching.
- Time complexity: $O(\text{nodes}(A) \cdot \text{nodes}(B))$
- Normalized version of the algorithm:

$$NSTM(A, B) = \frac{\text{SimpleTreeMatching}(A, B)}{(\text{nodes}(A) + \text{nodes}(B))/2}$$

The simple tree matching algorithm

Algorithm: SimpleTreeMatching(T' , T'')

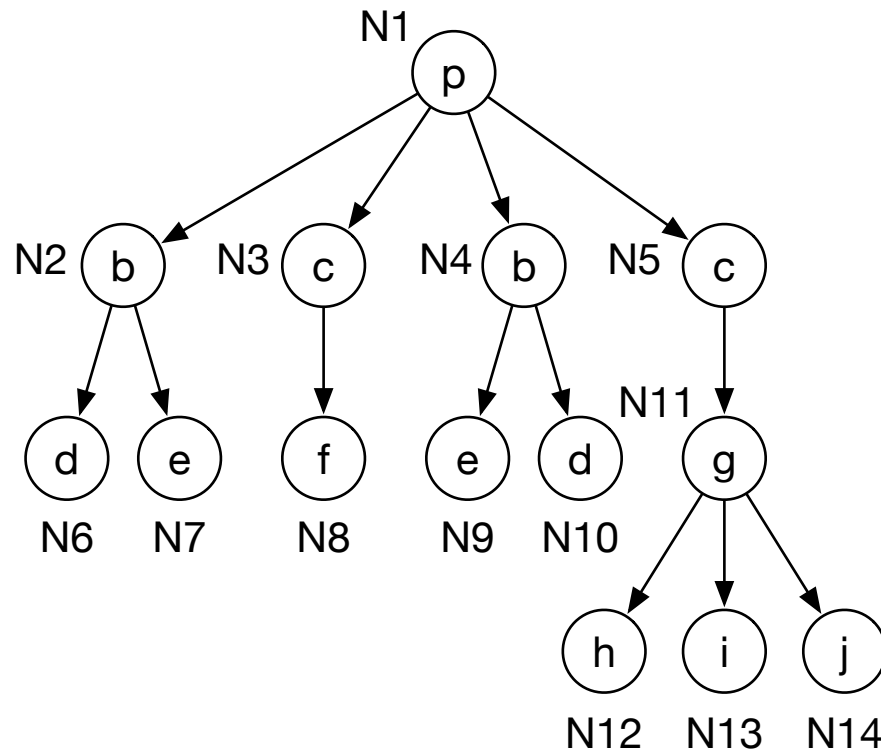
```
1 if  $T'$  has the same label of  $T''$  then  
2    $m \leftarrow d(T')$   
3    $n \leftarrow d(T'')$   
4   for  $i = 0$  to  $m$  do  
5      $M[i][0] \leftarrow 0$   
6   for  $j = 0$  to  $n$  do  
7      $M[0][j] \leftarrow 0$   
8   forall  $i$  such that  $1 \leq i \leq m$  do  
9     forall  $j$  such that  $1 \leq j \leq n$  do  
10       $W_{ij} = \text{SimpleTreeMatching}(T'(i-1), T''(j-1))$   
11       $M[i][j] \leftarrow \text{Max}(M[i][j-1], M[i-1][j], M[i-1][j-1] + W_{ij})$   
12 return  $M[m][n] + 1$   
13 else  
14 return 0
```

$d(n)$ – degree of node n

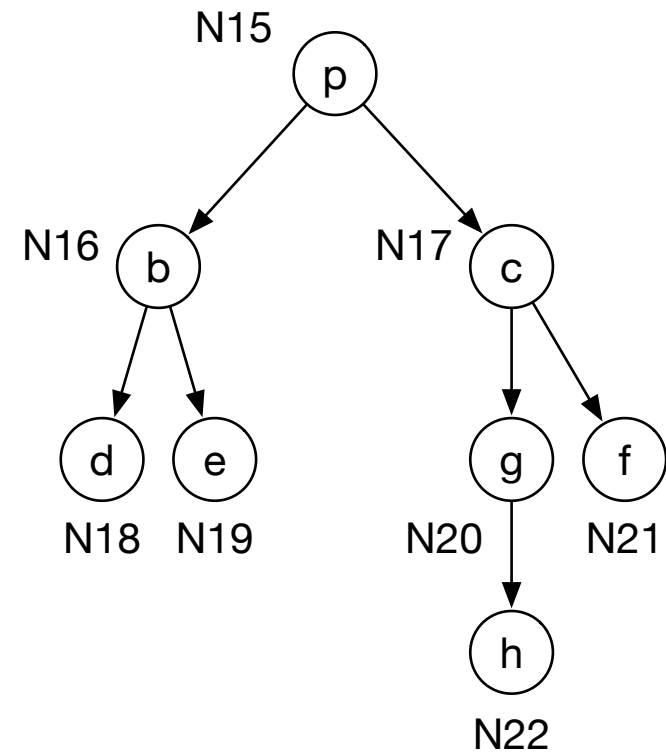
$T(i)$ – i -th subtree of node T

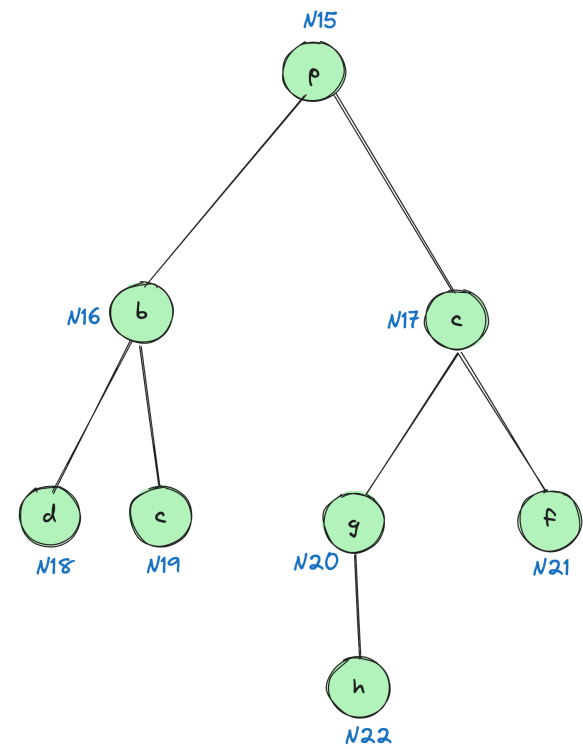
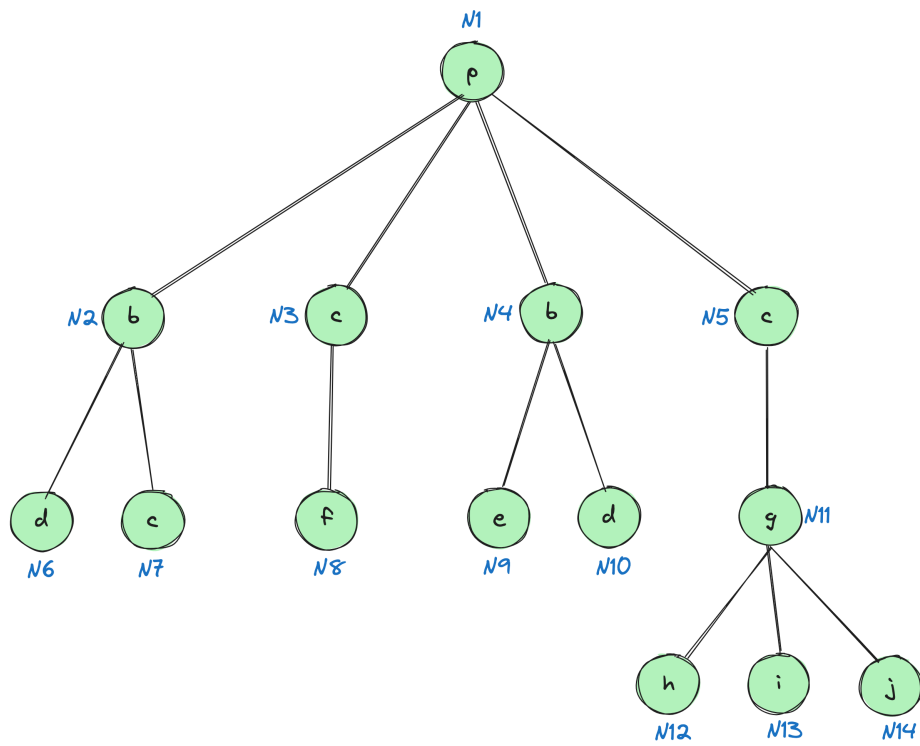
Tree matching algorithm - example

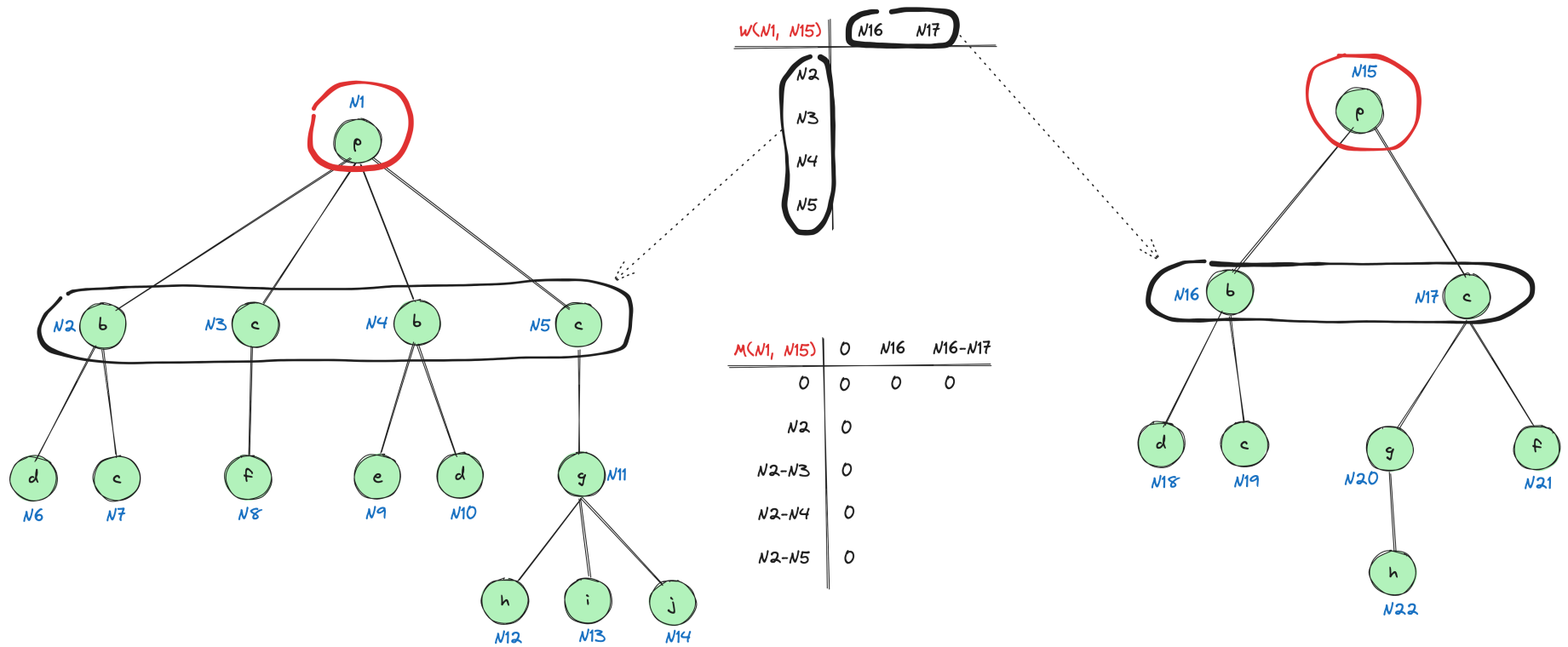
(A)

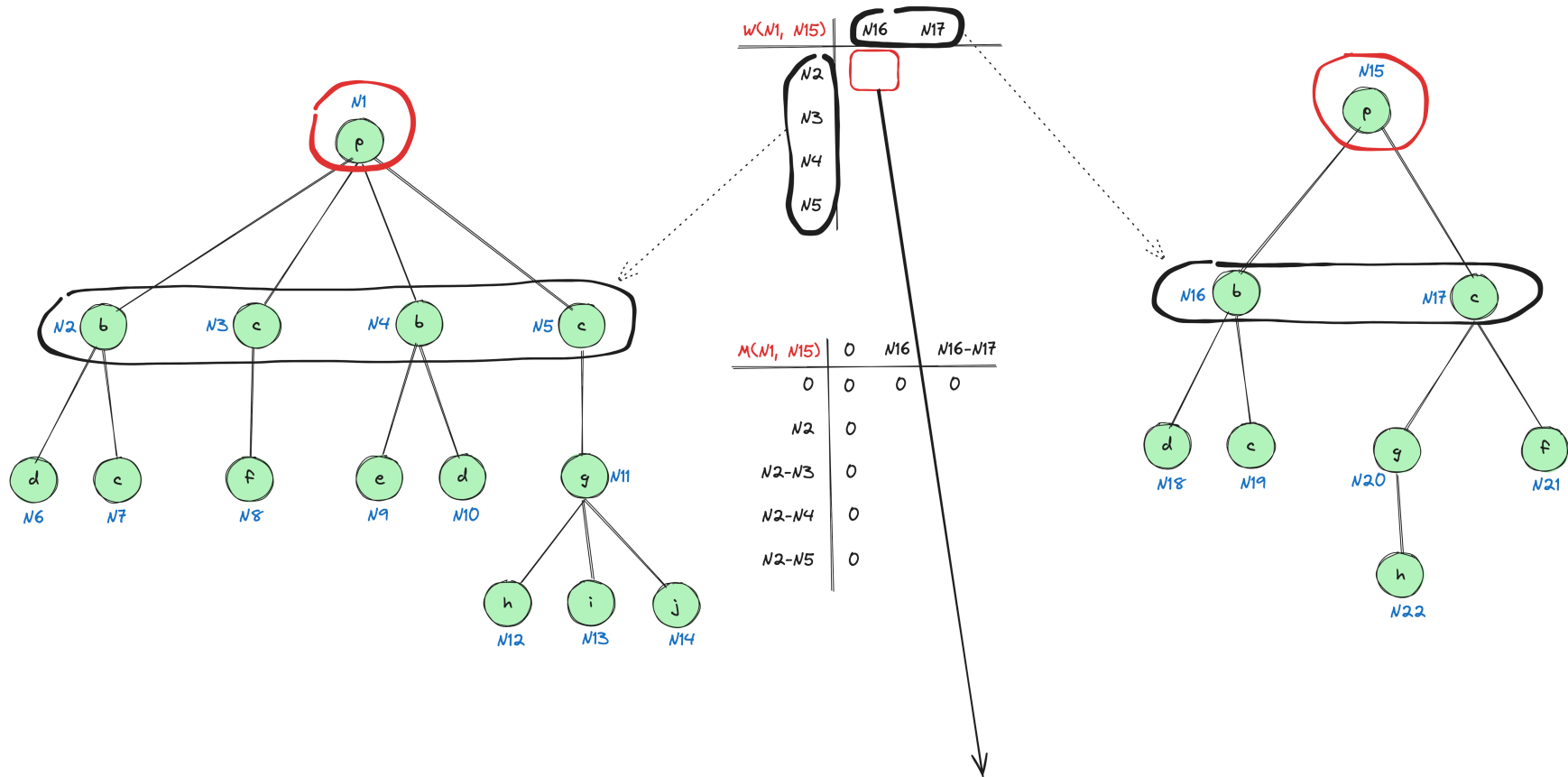


(B)









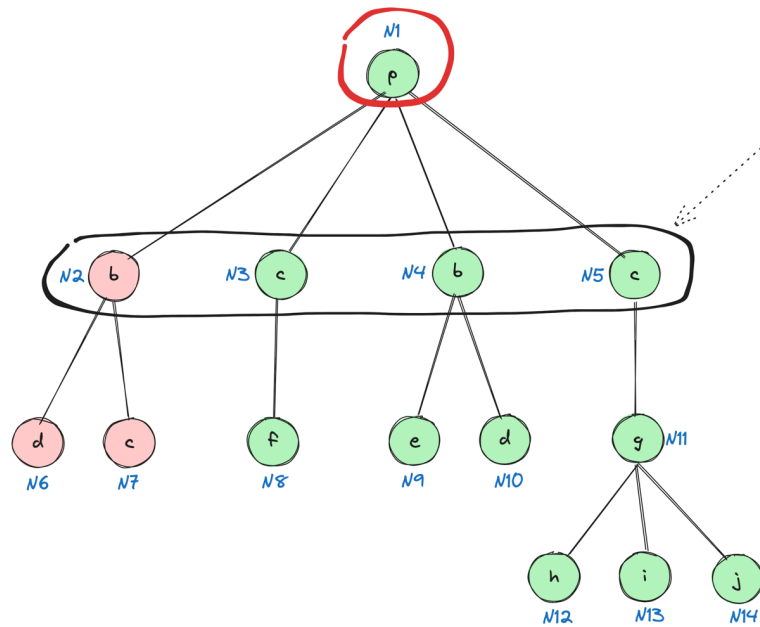
```

8  forall i such that  $1 \leq i \leq m$  do
9    forall j such that  $1 \leq j \leq n$  do
10      $W_{ij} = \text{SimpleTreeMatching}(T'(i-1), T''(j-1))$ 
11      $M[i][j] \leftarrow \text{Max}(M[i][j-1], M[i-1][j], M[i-1][j-1] + W_{ij})$ 
12  return  $M[m][n] + 1$ 

```

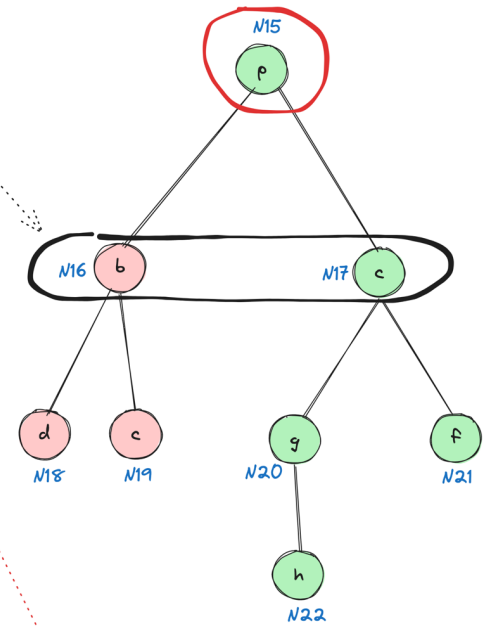
$W(N2, N16)$	$N18$	$N19$
$N6$		
$N7$		

$M(N2, N16)$	0	$N18$	$N18-N19$
0	0	0	0
$N6$	0		
$N6-N7$	0		



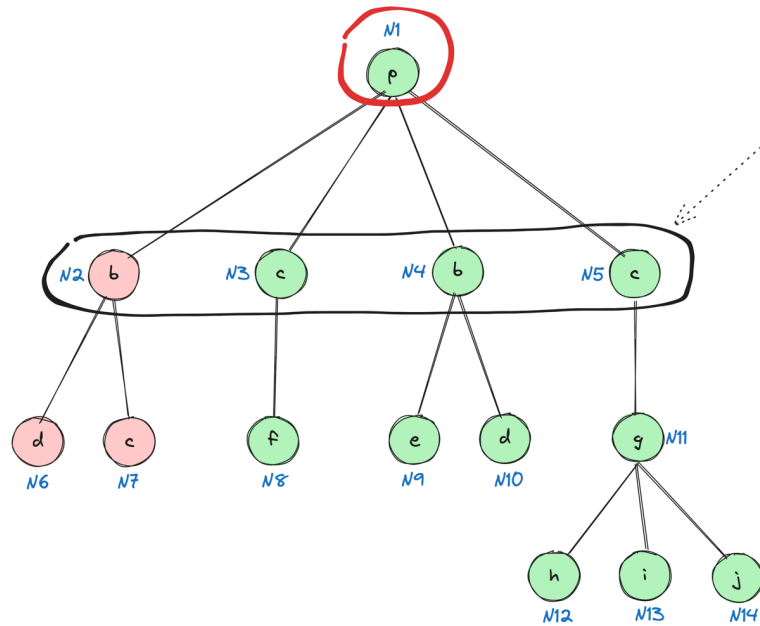
$w(N1, N15)$	N16	N17
N2	3	
N3		
N4		
N5		

$M(N1, N15)$	0	N16	N16-N17
0	0	0	0
N2	0		
N2-N3	0		
N2-N4	0		
N2-N5	0		



$w(N2, N16)$	N18	N19
N6	1	0
N7	0	1

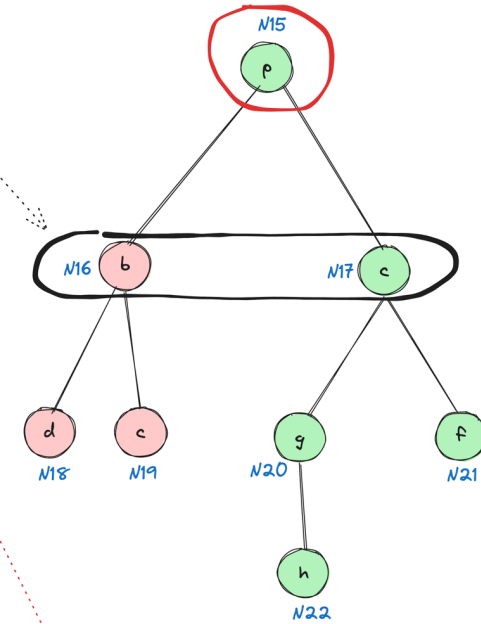
$M(N2, N16)$	0	N18	N18-N19
0	0	0	0
N6	0	1	1
N6-N7	0	1	2
			+ 1 = 3



$w(N1, N15)$	N16	N17
N2	3	0
N3	0	2
N4	2	0
N5	0	3

$M(N1, N15)$	0	N16	N16-N17
0	0	0	0
N2	0	3	3
N2-N3	0	3	5
N2-N4	0	3	5
N2-N5	0	3	6

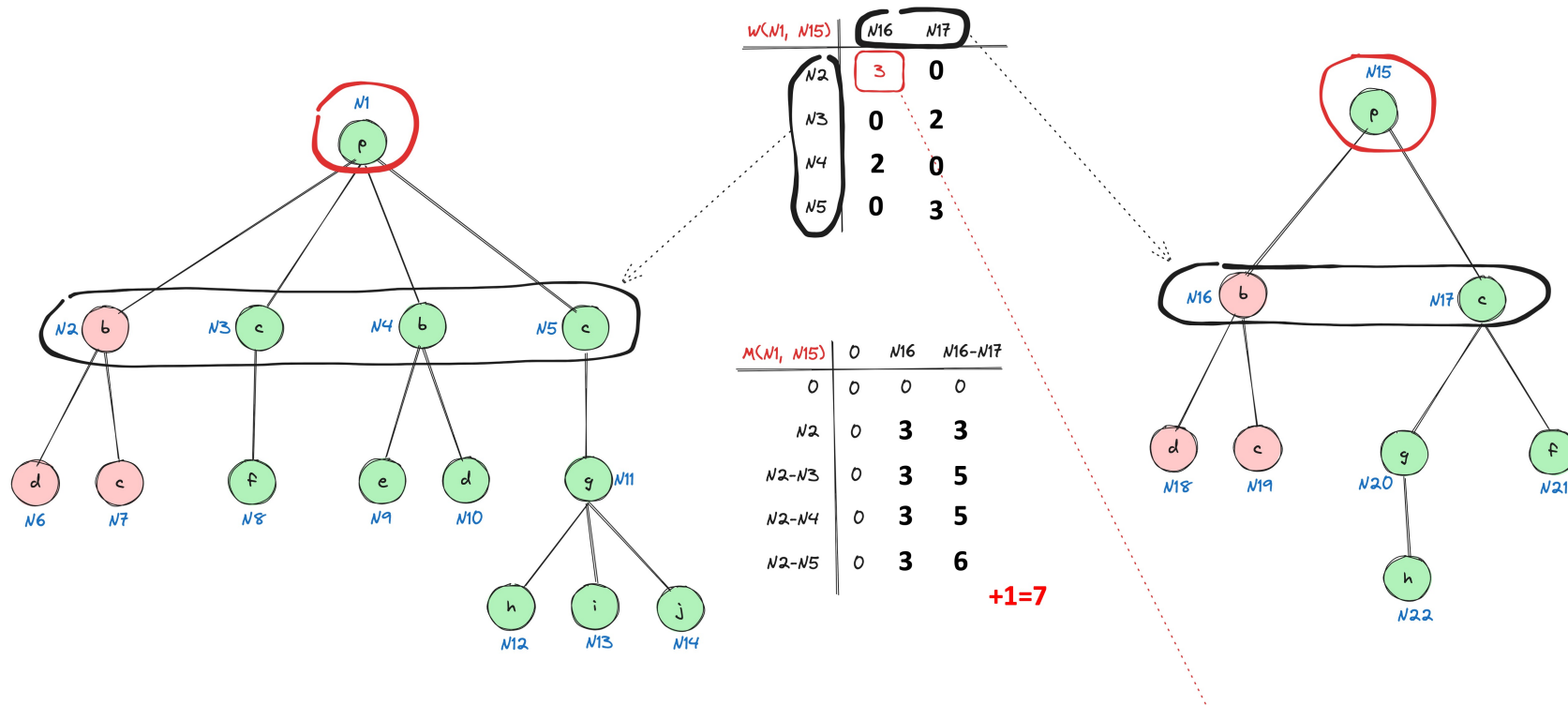
+1=7



$w(N2, N16)$	N18	N19
N6	1	0
N7	0	1

$M(N2, N16)$	0	N18	N18-N19
0	0	0	0
N6	0	1	1
N6-N7	0	1	2

+1=3

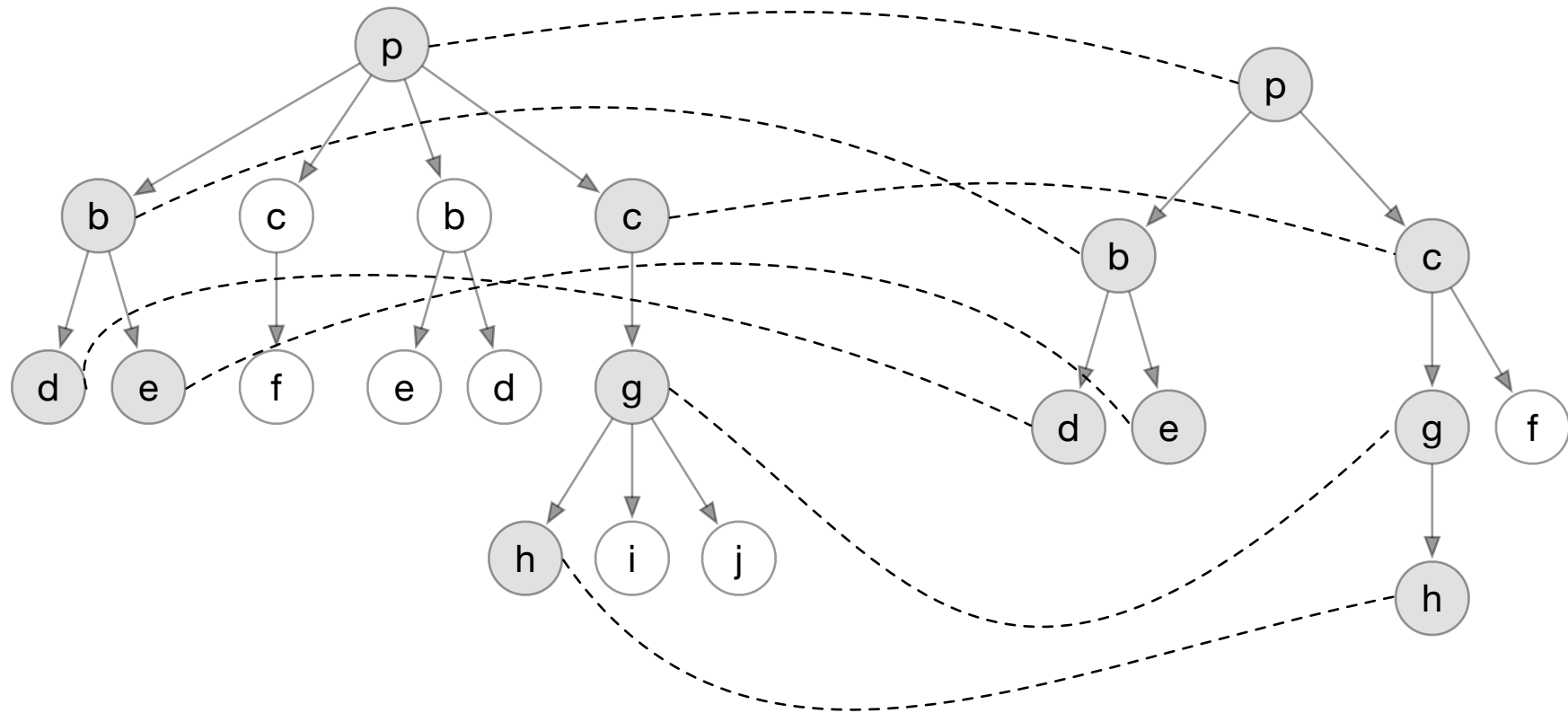


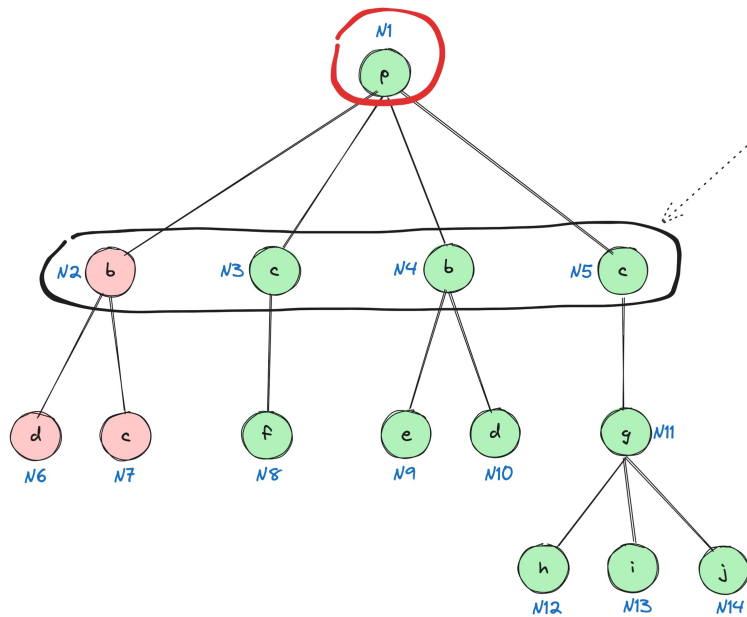
$$NSTM(A, B) = \frac{\text{SimpleTreeMatching}(A, B)}{(\text{nodes}(A) + \text{nodes}(B))/2} = \frac{7}{(14 + 8)/2} = \frac{7}{11} = 0.64$$

Tree matching algorithm - example

(A)

(B)

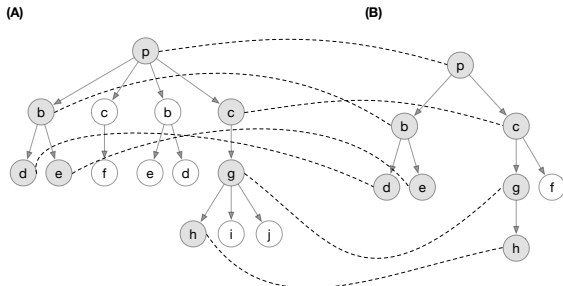
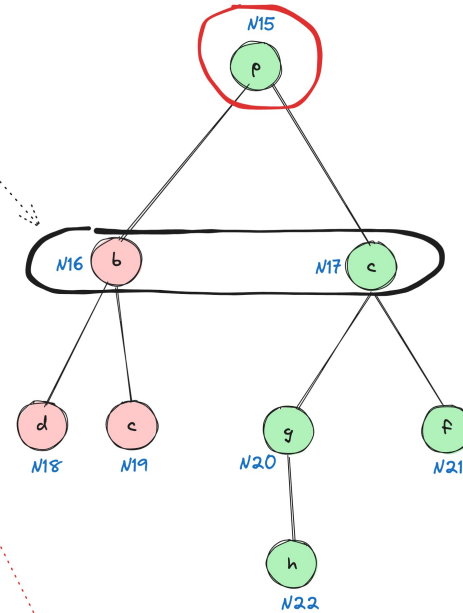




$w(N1, N15)$	N16	N17
N2	3	0
N3	0	2
N4	2	0
N5	0	3

$M(N1, N15)$	0	N16	N16-N17
0	0	0	0
N2	0	3	3
N2-N3	0	3	5
N2-N4	0	3	5
N2-N5	0	3	6

+1=7



```

8 forall i such that  $1 \leq i \leq m$  do
9   forall j such that  $1 \leq j \leq n$  do
10     $W_{ij} = \text{SimpleTreeMatching}(T'(i-1), T''(j-1))$ 
11     $M[i][j] \leftarrow \text{Max}(M[i][j-1], M[i-1][j], M[i-1][j-1] + W_{ij})$ 
12 return  $M[m][n] + 1$ 

```

The simple tree matching algorithm

■ Limitations:

- cannot match permutation of nodes.
- no level crossing is allowed (impossible to match nodes at different levels).

Benefits:

- easy to implement.
- efficient when applied to HTML trees.
- widely used.

The weighted tree matching algorithm

Algorithm: WeightedTreeMatching(T' , T'')

The code for the algorithm is similar to the *SimpleTreeMatching* algorithm, where you change the line 12 with the following code:

```
1 if  $m > 0$  AND  $n > 0$  then  
2   return  $M[m][n] * \frac{1}{\text{Max}(t(T'), t(T''))}$   
3 else  
4   return  $M[m][n] + \frac{1}{\text{Max}(t(T'), t(T''))}$ 
```

```
8 forall  $i$  such that  $1 \leq i \leq m$  do  
9   forall  $j$  such that  $1 \leq j \leq n$  do  
10     $W_{ij} = \text{SimpleTreeMatching}(T'(i-1), T''(j-1))$   
11     $M[i][j] \leftarrow \text{Max}(M[i][j-1], M[i-1][j], M[i-1][j-1] + W_{ij})$   
12  return  $M[m][n] + 1$ 
```

- $t(n)$ – number of total siblings of node n including itself.

Multiple alignment

- Finding repeated patterns from HTML strings / pages
- Approaches
 - Optimal multiple alignment (Carrillo and Lipman, 1988)
 - Center star method (Gusfield, 1997)
 - Partial tree alignment method (Zhai and Liu, 2005)
 - ...

Center star method

- Classic technique
- Used for multiple string alignment, can be adopted to trees.
- Let $d(s_c, s_i)$ be the distance of two strings and S the set of strings for alignment. Then a string s_c - center string is selected as:
$$s_c = \arg \min_{s_c} \sum_{s_i \in S} d(s_c, s_i)$$
- The algorithm iteratively computes the alignment of other strings with s_c .

Center star method

Algorithm: CenterStar(S)

- 1 choose the center star s_c
- 2 $M = List(s_c)$
- 3 **for** each s in $S - \{s_c\}$ **do**
- 4 $c* =$ aligned version of $s_c \in M$
- 5 $s', c*' =$ optimally aligned strings of s and $c*$
- 6 update strings in M with spaces where added to $c*$
- 7 $M.add(s')$
- 8 $M.add(c*')$
- 9 **return** multiple string alignment M

Center star method

- Time complexity: $O(k^2 n^2)$
- Shortcomings
 - slow for pages with many data records / many tags
 - if s_c does not have a particular data item, other records might not be properly aligned

Partial tree alignment

- Used for multiple tree alignment
- Idea
 - progressively growing a *seed tree* T_s
 - for each T_i ($i \neq s$) find matching nodes otherwise insert (*)
- **Partial** – if insertion cannot be done, it remains unmatched

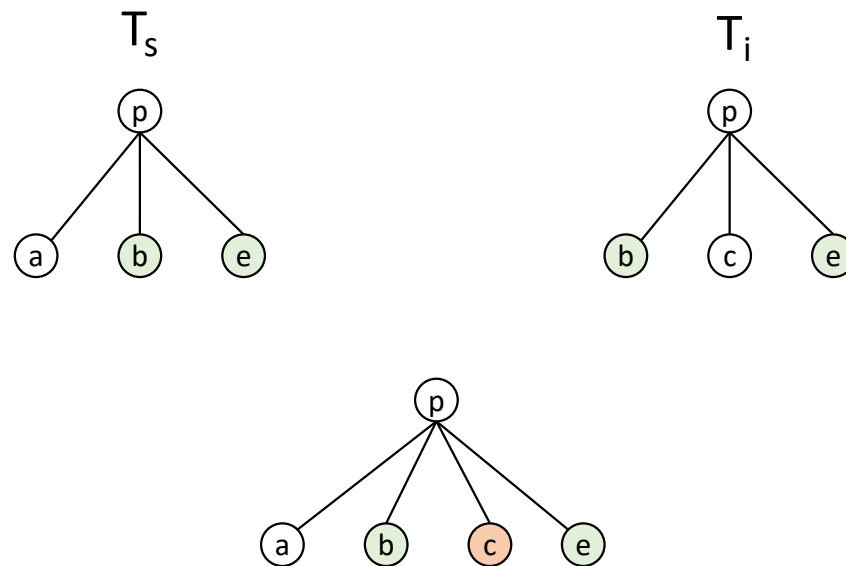
Partial tree alignment – two trees

- Some nodes in T_i can be matched to nodes in T_s
- Non-matched nodes from T_i can contain optional items, thus v_i is inserted into T_s
 - Insertion only if a location in T_s can be *uniquely determined*
- Let $v_j \dots v_m$ be unmatched consecutive sibling nodes from T_i :
 - Parent of $v_j \dots v_m$ has a match in T_s , where nodes should be inserted
 - Inserted only if position can be uniquely determined, otherwise not inserted

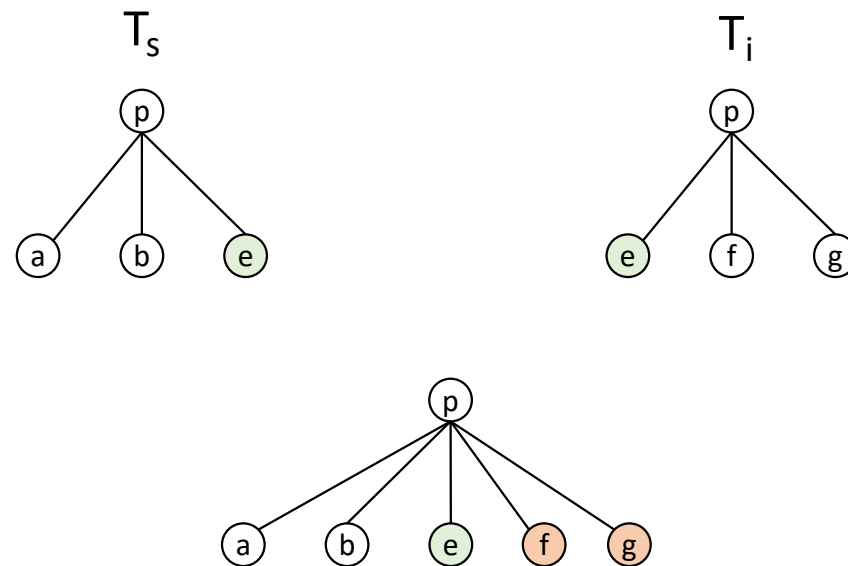
Partial tree alignment – two trees

- Location for inserting $v_j \dots v_m$ from T_i to T_s uniquely decided if:
 - If $v_j \dots v_m$ have two neighboring siblings in T_i , one on the right and one on the left, that are matched with two consecutive siblings in T_s .
 - If $v_j \dots v_m$ has only one left neighboring sibling x in T_i and x matches the right most node x in T_s , then $v_j \dots v_m$ can be inserted after node x in T_s .
 - If $v_j \dots v_m$ has only one right neighboring sibling x in T_i and it matches the left most node x in T_s , then $v_j \dots v_m$ can be inserted before node x in T_s .

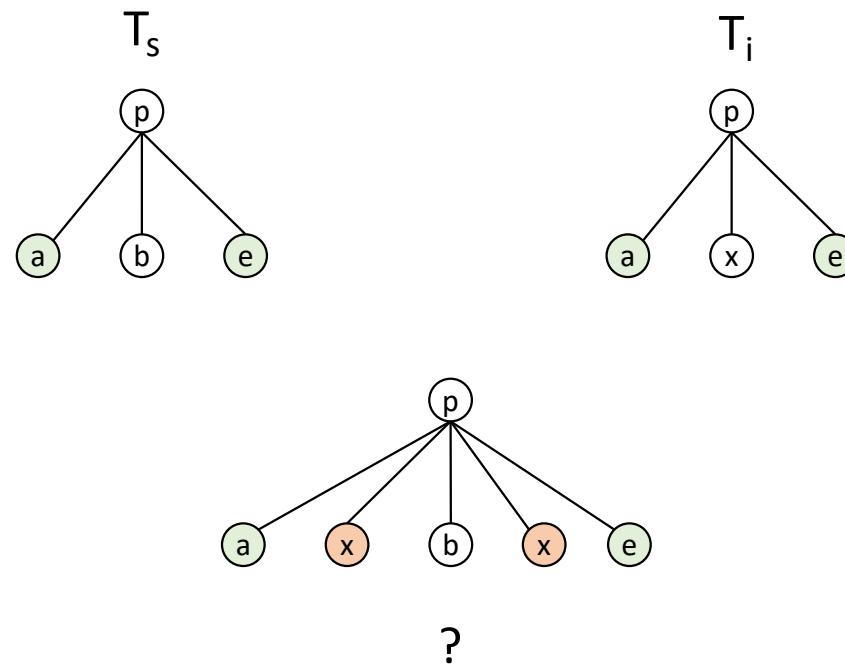
Partial tree alignment – two trees (demo)



Partial tree alignment – two trees (demo)



Partial tree alignment – two trees (demo)



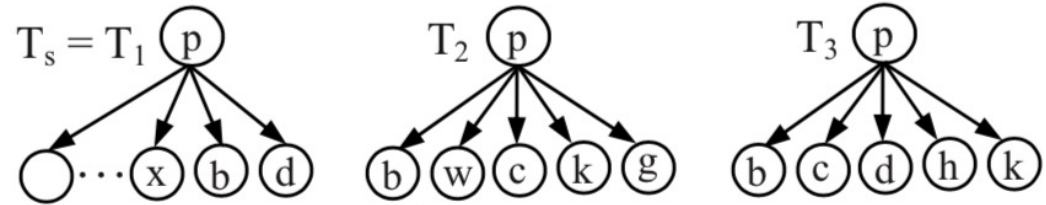
Partial tree alignment – multiple trees

Algorithm: PartialTreeAlignment(S)

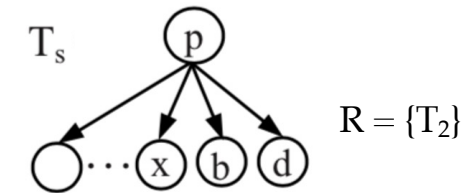
```
1 Sort trees in  $S$  in descending order of the number of data items
2  $T_s \leftarrow$  select and delete the first tree from  $S$ 
3  $R \leftarrow \emptyset$ 
4 while  $S \neq \emptyset$  do
5    $T \leftarrow$  select and delete next tree from  $S$ 
6   STM( $T_s, T_i$ )
7   AlignTrees( $T_s, T_i$ )
8   if  $T_i$  is not completely aligned with  $T_s$  then
9     if InsertIntoSeed( $T_s, T_i$ ) then
10        $S \leftarrow S \cup R$ 
11        $R \leftarrow \emptyset$ 
12     if still unaligned items in  $T_i$  that are not inserted into  $T_s$  then
13        $R \leftarrow R \cup \{T_i\}$ 
14 if  $R \neq \emptyset$  then
15   PartialTreeAlignment( $R$ )
16 return data table of alignment results for each  $T_i$ 
```

Partial tree alignment – multiple trees (demo)

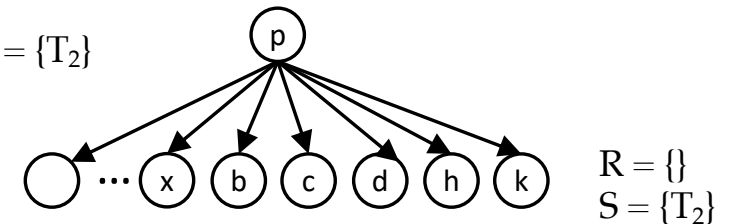
Initial set S



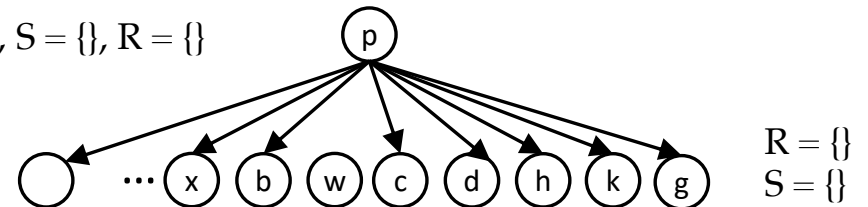
$S = \{T_2, T_3\}, R = \{\}$
 Iteration 1:
 $T = T_2, S = \{T_3\}$



Iteration 2:
 $T = T_3, S = \{\}, R = \{T_2\}$



Iteration 3:
 $T = T_2, S = \{\}, R = \{\}$



Algorithm: PartialTreeAlignment(S)

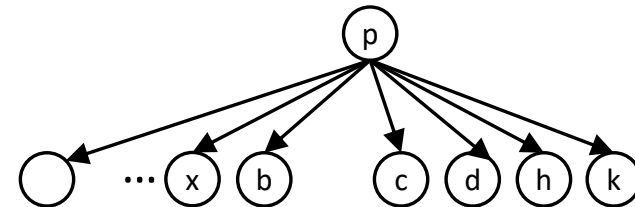
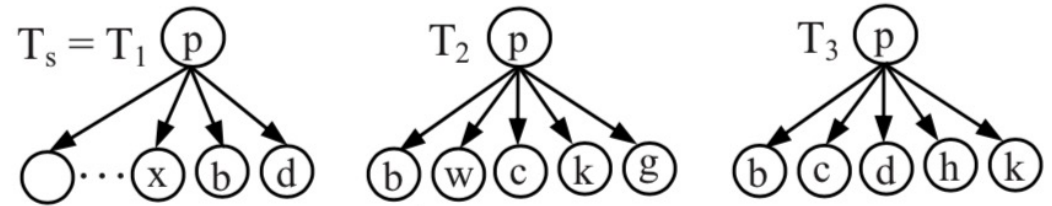
```

1 Sort trees in  $S$  in descending order of the number of data items
2  $T_s \leftarrow$  select and delete the first tree from  $S$ 
3  $R \leftarrow \emptyset$ 
4 while  $S \neq \emptyset$  do
5    $T \leftarrow$  select and delete next tree from  $S$ 
6   STM( $T_s, T_i$ )
7   AlignTrees( $T_s, T_i$ )
8   if  $T_i$  is not completely aligned with  $T_s$  then
9     if InsertIntoSeed( $T_s, T_i$ ) then
10        $S \leftarrow S \cup R$ 
11        $R \leftarrow \emptyset$ 
12   if still unaligned items in  $T_i$  that are not inserted into  $T_s$  then
13      $R \leftarrow R \cup \{T_i\}$ 
14 if  $R \neq \emptyset$  then
15   PartialTreeAlignment( $R$ )
16 return data table of alignment results for each  $T_i$ 
```

Partial tree alignment - multiple trees

(DEMO)

Initial set S



$R = \{\}$
 $S = \{\}$

	...	x	b	w	c	d	h	k	g
T_1	...	1	1			1			
T_2			1	1	1			1	1
T_3			1		1	1	1	1	

Building DOM/tag trees

- Necessary for many data extraction algorithms
- Approaches:
 - Using tags only
 - Using tags and visual cues

Building DOM/tag trees (example 1)

- Tags example

```
1  <table border="2">
2    <tr>
3      <td> data1 <td>
4      </td> data2 </td>
5    <tr>
6      <td> data3 </td>
7      <td> data4
8    <tr>
9  </table>
```

Building DOM/tag trees (example 1)

- Tags example

```
1  <table border="2">
2    <tr>
3      <td> data1 <td>
4      </td> data2 </td>
5    <tr>
6      <td> data3 </td>
7      <td> data4
8    <tr>
9  </table>
```

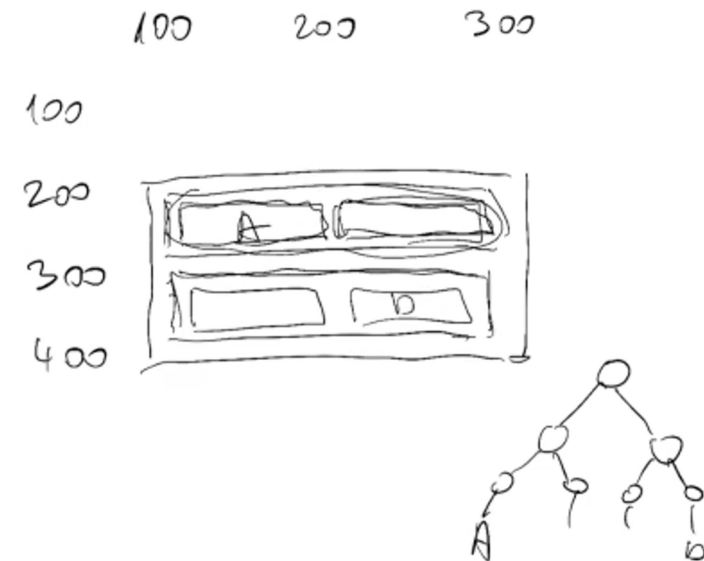
data2

data1	
data3	data4

Building DOM/tag trees (example 2)

- Visual cues example

left	right	top	bottom
100	300	200	400
100	300	200	300
100	200	200	300
200	300	200	300
100	300	300	400
100	200	300	400
200	300	300	400



Data extraction – single list page

- A list (a data region) containing data records (without nesting)

- Process:
 1. Identify a list (i.e. *data region*)
 2. Segment data records in each list
 3. Align data items within data records

1.



Customer
Rating:
★★★★☆

Apple iBook Notebook M8600LL/A (600-MHz PowerPC G3, 128 MB RAM, 20 GB hard drive)

Buy new: **\$1,194.00**

Usually ships in 1 to 2 days

Best use: (what's this?)	Business: ●●●●○	Portability: ●●●●●	Desktop Replacement: ●●●●○	Entertainment: ●●●●○
--	--------------------	-----------------------	-------------------------------	-------------------------

600 MHz PowerPC G3, 128 MB SRAM, 20 GB Hard Disk, 24x CD-ROM, AirPort ready, and Mac OS X, Mac OS X, Mac OS 9.2, Quick Time, iPhoto, iTunes 2, iMovie 2, AppleWorks, Microsoft IE

2.



Customer
Rating:
★★★★☆

Apple Powerbook Notebook M8591LL/A (667-MHz PowerPC G4, 256 MB RAM, 30 GB hard drive)

Buy new: **\$2,399.99**

Best use: (what's this?)	Portability: ●●●●○	Desktop Replacement: ●●●●○	Entertainment: ●●●●○
--	-----------------------	-------------------------------	-------------------------

667 MHz PowerPC G4, 256 MB SDRAM, 30 GB Ultra ATA Hard Disk, 24x (read), 8x (write) CD-RW, 8x; included via combo drive DVD-ROM, and Mac OS X, QuickTime, iMovie 2, iTunes(6), Microsoft Internet Explorer, Microsoft Outlook Express, ...

1.



Customer
Rating:
★★★★☆

Apple iBook Notebook M8600LL/A (600-MHz PowerPC G3, 128 MB RAM, 20 GB hard drive)

Buy new: **\$1,194.00**

Usually ships in 1 to 2 days

Best use: ([what's this?](#))

Business: ●●●○○

Portability: ●●●●●

R

600 MHz PowerPC G3, 128 MB SRAM, 20 GB Hard Drive, and Mac OS X, Mac OS X, Mac OS 9.2, Quick Time, iPhoto 2, AppleWorks, Microsoft IE

2.



Customer
Rating:
★★★★☆

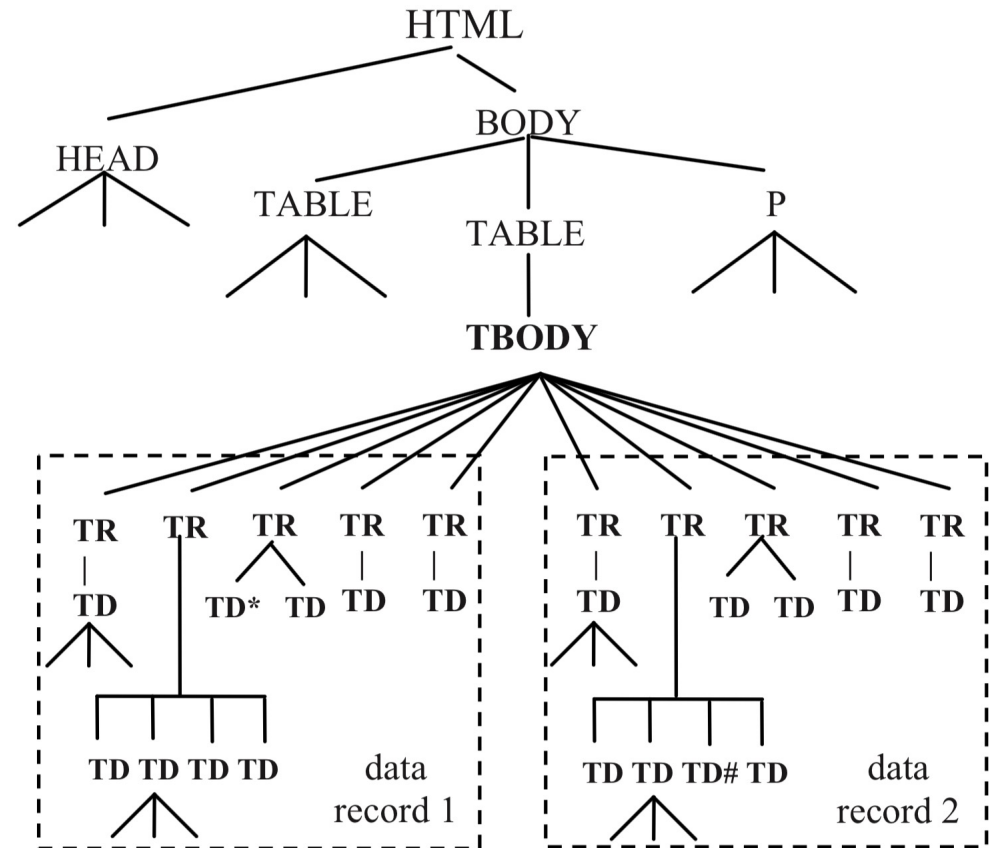
Apple Powerbook Notebook M8591LL/A (667-MHz PowerPC G4, 256 MB RAM, 30 GB hard drive)

Buy new: **\$2,399.99**

Best use: ([what's this?](#))

Portability: ●●●○○ Desktop Replacement: ●●●●●

667 MHz PowerPC G4, 256 MB SDRAM, 30 GB Ultra ATA/100 (write) CD-RW, 8x; included via combo drive DVD-R, iMovie 2, iTunes(6), Microsoft Internet Explorer, Microsoft



Rezultati iskanja: mleko

- Kozmetika in higiena (406)
- Nega telesa (237)
- Nega obraza (154)
- Nega las (15)
- Priprava napitkov (44)
- Penilci mleka (38)
- Kavni aparati (4)
- Grelniki vode (1)
- Posode za napitke (1)
- Parfumerija (31)
- Parfumi (25)
- Toaletne vode (6)
- Hranjenje (27)
- Hrana za otroke (21)
- Otroški jedilni pribor (5)
- Otroške stekleničke (1)
- Kuhinjska posoda (19)
- Lonci in kozice (16)
- Posode za tople napitke (3)
- Mačke (11)
- Čiščenje (10)
- Otroška kozmetika (10)

Cena

MIN

MAX

644 izdelkov

Priljubljenost



CLATRONIC penilec mleka MS 3326

52,70 €
v mimovrste=)

V trgovino

Primerjaj z ostalimi



VAVA penilnik mleka

od 37,90 €
v 3 trgovinah

Več o ponudbi

Primerjaj z ostalimi



ADLER penilec mleka (AD4478)

od 28,30 €
v 2 trgovinah

Več o ponudbi

Primerjaj z ostalimi



MIA penilnik mleka Cappuccino Mix MF 3822 3V1, 580ml

od 34,33 €
v 2 trgovinah

Več o ponudbi



SEVERIN penilec mleka SM 3590

od 8,95 €
v 2 trgovinah

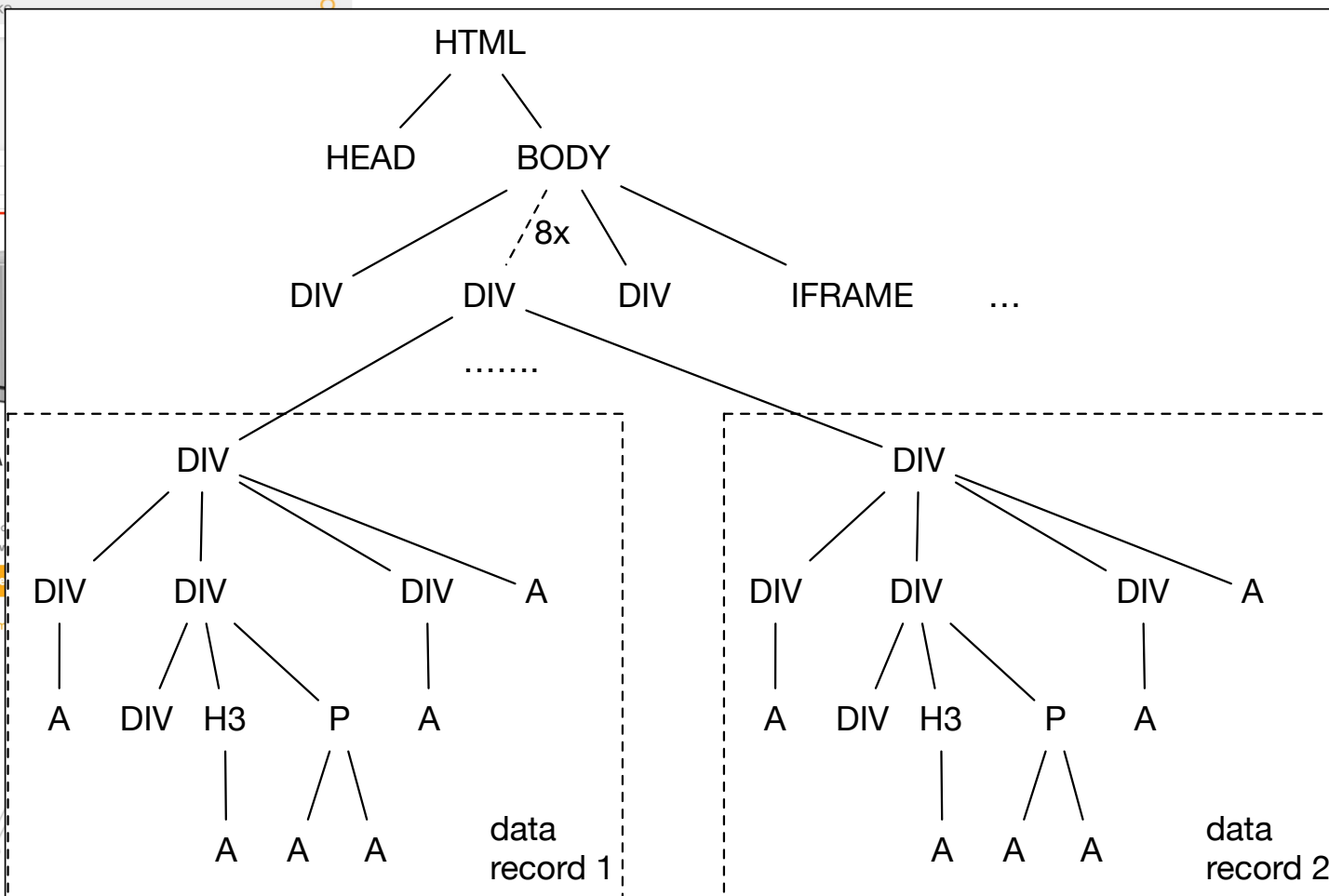
Več o ponudbi



PHILIPS penilec mleka CA6500/10

68,00 €
v Electronic4You - Majdič

V trgovino

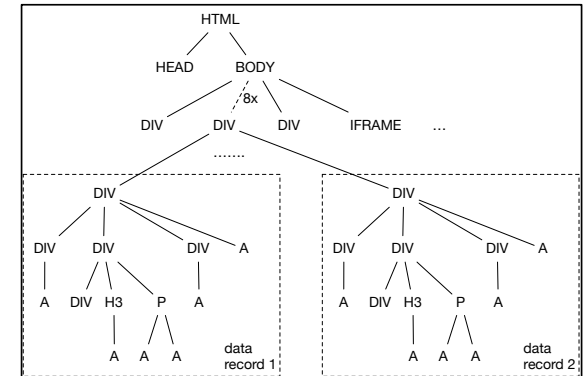


Two observations about data records

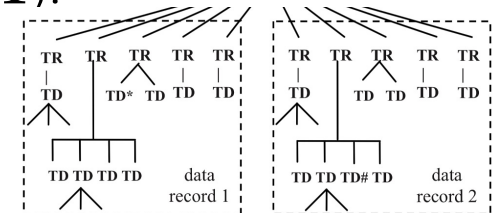
- Data records use *the same* HTML mark-up encoding
- Problems:
 - efficiency (should not try all possibilities)
 - tags matching
- Two observations to solve the problems (Liu, Grossman, Zhai, 2003)

Mining data regions

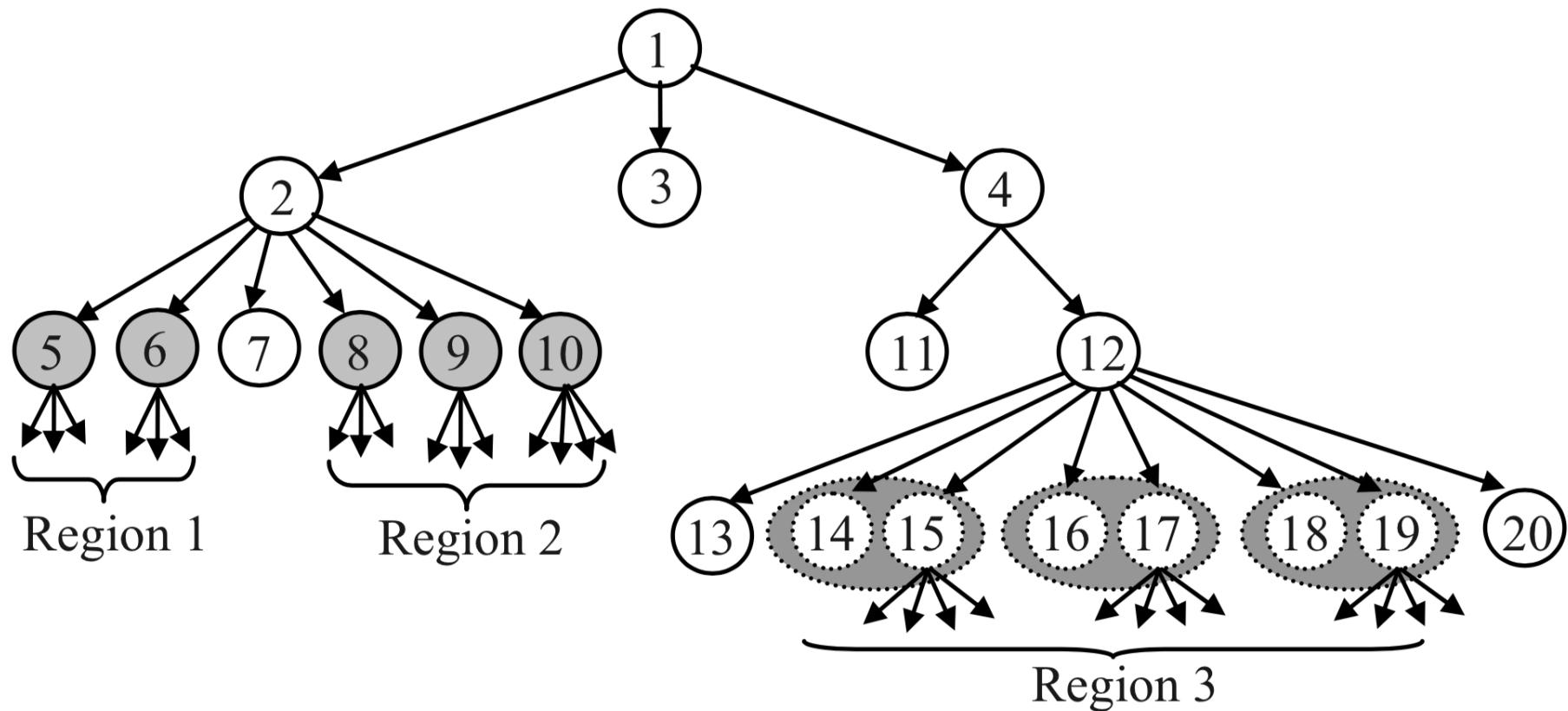
- Mining data regions containing a list of data records



- A *generalized node* (node combination) of length r ($r \geq 1$):
 - the nodes have the same parent
 - the nodes are adjacent
- A *data region* – a collection of two or more generalized nodes:
 - the generalized nodes have the same parent
 - the generalized nodes have the same length
 - the generalized nodes are adjacent
 - the similarity between generalized nodes is greater than a fixed threshold



Mining data regions

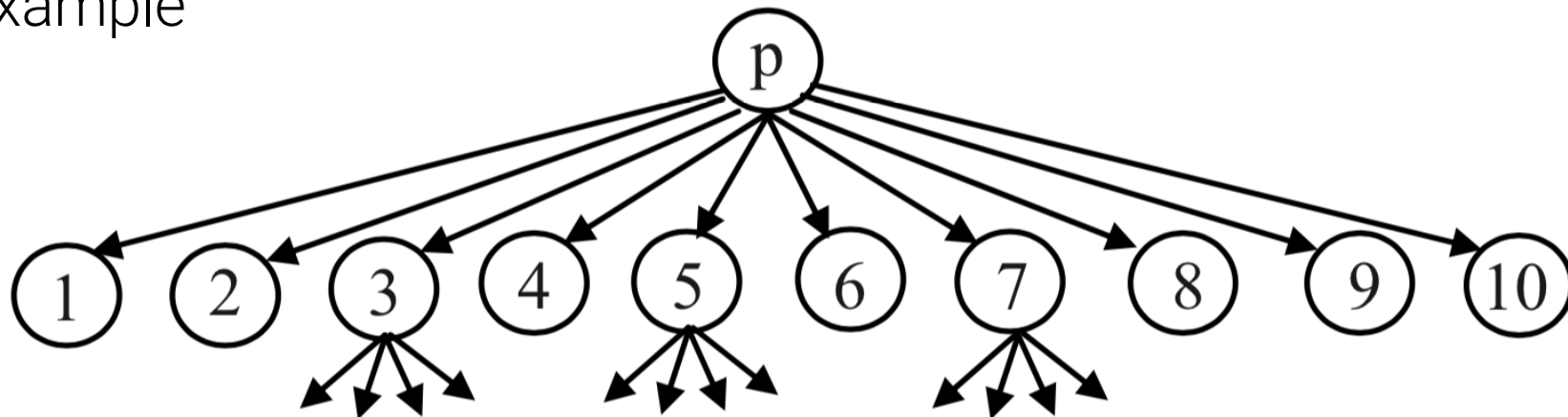


Generalized nodes comparison

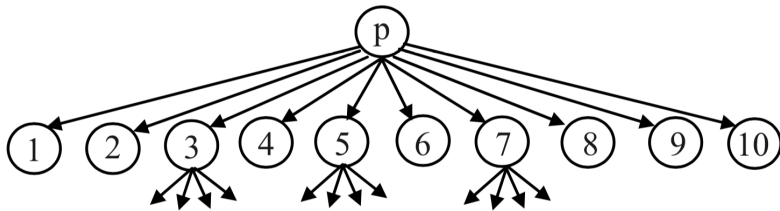
■ Goals:

- Where does a first generalized node start?
- How many consecutive tags does a generalized node have?

■ Example



Generalized nodes comparison (example)



Example:

Let $K=3$

Comparisons (based on $r=1..3$):

Start from node 1:

(1,2), (2,3), (3,4), (4,5), (5,6), (6,7), (7,8), (8,9), (9,10)

(1-2, 3-4), (3-4, 5-6), (5-6, 7-8), (7-8, 9-10)

(1-2-3, 4-5-6), (4-5-6, 7-8-9)

Start from node 2:

(2-3, 4-5), (4-5, 6-7), (6-7, 8-9)

(2-3-4, 5-6-7), (5-6-7, 8-9-10)

Start from node 3:

(3-4-5, 6-7-8)

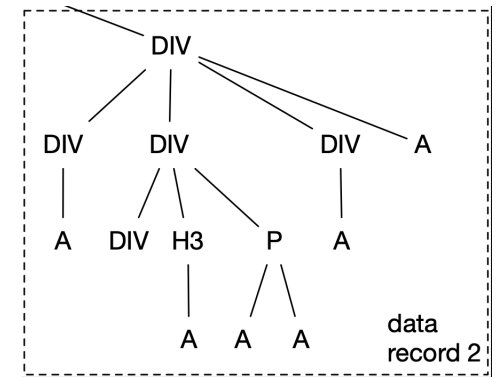
→ Total 21 comparisons.

Mining data regions algorithm

Algorithm $\text{MDR}(\text{Node}, K, \tau)$

```

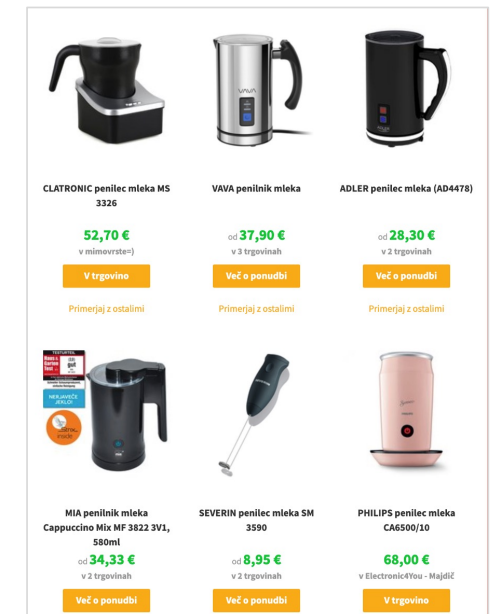
1  if  $\text{TreeDepth}(\text{Node}) \geq 3$  then
2       $\text{CombComp}(\text{Node.Children}, K);$ 
3       $\text{DataRegions} \leftarrow \text{IdenDRs}(\text{Node}, K, \tau);$ 
4      if  $(\text{UncoveredNodes} \leftarrow \text{Node.Children} - \bigcup_{DR \in \text{DataRegions}} DR) \neq \emptyset$  then
5          for each  $\text{ChildNode} \in \text{UncoveredNodes}$  do
6               $\text{DataRegions} \leftarrow \text{DataRegions} \cup \text{MDR}(\text{ChildNode}, K, \tau);$ 
7      return  $\text{DataRegions}$ 
8  else return  $\emptyset$ 
    
```



- Number of string/tree comparisons - $O(nk^2)$, n – number of tree nodes

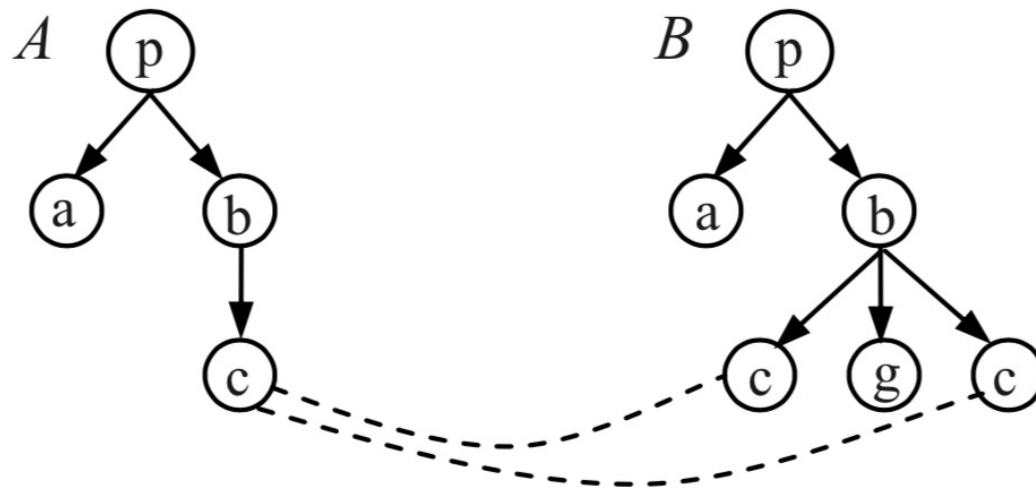
Data records within data regions

- „If a generalized node contains two or more data records, these data records must be similar in terms of their tag strings.“
- Data records identification:
 1. Produce one-rooted tree for each generalized node
 2. Run MDR to retrieve data records



Data item alignment and extraction

- Producing an extraction pattern within a data region:
 - Produce a rooted tree for each data record
 - Align the resulting trees
- Conflict resolution (Zhai and Liu, 2006)



Extraction Based on a Single List Page: Nested Data Records

- The number of elements in a list of each data record can be different – previous algorithms will not work due to the fixed threshold problem!

The screenshot displays the website of the Faculty of Computer and Data Science. It features a navigation bar with links to Home, Departments, Admissions, Research, Curriculum, and Contact. The main content is divided into two sections: Contact and Curriculum Overview. The Contact section provides email, phone, office location, and office hours. The Curriculum Overview section lists two degree programs: Bachelor of Computer Science (t1) and Bachelor of Data Science (t2). Each program is represented by a list of courses with their respective semesters and lecturers.

Program	Course ID	Course Name	Semester	Lecturer
Bachelor of Computer Science (t1)	N1	Introduction to Programming (13)	Semester: 1 (14)	Prof. John Smith (15)
	N3	Data Structures (16)	Semester: 2 (17)	Dr. Laura Chen (18)
	N5	Fundamentals of Statistics (19)	Semester: 1 (110)	Dr. Marcus Lee (111)
	N7	Data Visualization (112)	Semester: 2 (113)	Lecturer not yet assigned
Bachelor of Data Science (t2)	N8	Machine Learning (114)	Semester: 3 (115)	Prof. Elena Rossi (116)

Hours

9:00–17:00

Curriculum Overview

Bachelor of Computer Science (t1)



N1

Course: Introduction to Programming (t3)

Semester: 1 (t4)

N2



Lecturer: Prof. John Smith (t5)



N3

Course: Data Structures (t6)

Semester: 2 (t7)

N4



Lecturer: Dr. Laura Chen (t8)

Bachelor of Data Science (t2)



N5

Course: Fundamentals of Statistics (t9)

Semester: 1 (t10)

N6



Lecturer: Dr. Marcus Lee (t11)



N7

Course: Data Visualization (t12)

Semester: 2 (t13)



Lecturer not yet assigned



N8

Course: Machine Learning (t14)

Semester: 3 (t15)

N9



Lecturer: Prof. Elena Rossi (t16)

University Curriculum Overview

Study Program T1

N1

Course: t3 Semester: t4

N2

Lecturer: t5

N3

Course: t6 Semester: t7

N4

Lecturer: t8

Study Program T2

N5

Course: t9 Semester: t10

N6

Lecturer: t11

N7

Course: t12 Semester: t13

N8

Course: t14 Semester: t15

N9

Lecturer: t16

Study programs: t1, t2

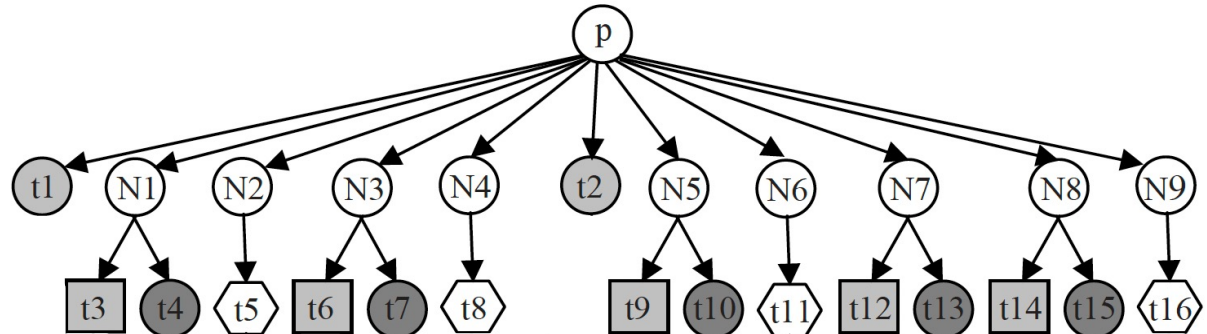
Courses: N1, N3, N5, N7, N8

Courses names: t3, t9, t12, t14

Semesters: t4, t7, t10, t13, t15

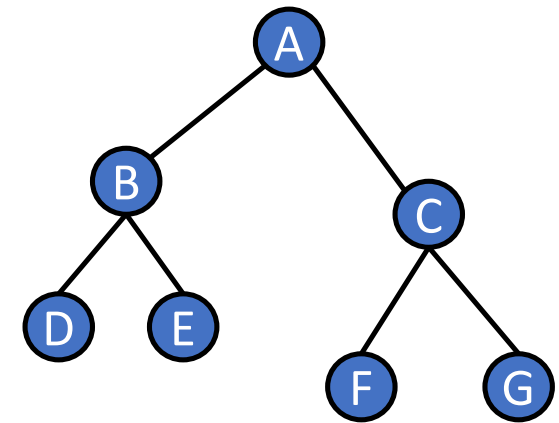
Lecturers: N2, N4, N6, N9

Official hours (per lecturer): t5, t8, t11, t16



Approach

- Traversing the DOM tree post-order (nested lists at lower levels are found first based on repeated patterns before going to higher levels).
- When a nested list is found, its records are collapsed to produce a single pattern which replaces the list of data records.
- When comparisons are made at a higher level, the algorithm only sees the pattern. Thus, it is treated as a flat data record.
- This solves the fixed threshold problem.



NET system

- Liu, B. and Y. Zhai. NET - A System for Extracting Web Data from Flat and, Nested Data Records. In Proceedings of Intl. Conf. on Web Information, Systems Engineering (WISE2005), 2005.

Algorithm NET(*Root*, τ)

```

1  TraverseAndMatch(Root,  $\tau$ );
2  for each top level node Node whose children have aligned data records do
3    PutDataInTables(Node); Returns relational tables (one for each data region)
4  endfor

```

TraverseAndMatch() must save the nodes whose children form data records.

Function TraverseAndMatch (*Node*, τ) post-order tree traversal algorithm.

```

1  if Depth(Node)  $\geq 3$  then
2    for each Child  $\in$  Node.Children do
3      TraverseAndMatch(Child,  $\tau$ );
4    endfor
5    Match(Node,  $\tau$ );
6  endif

```

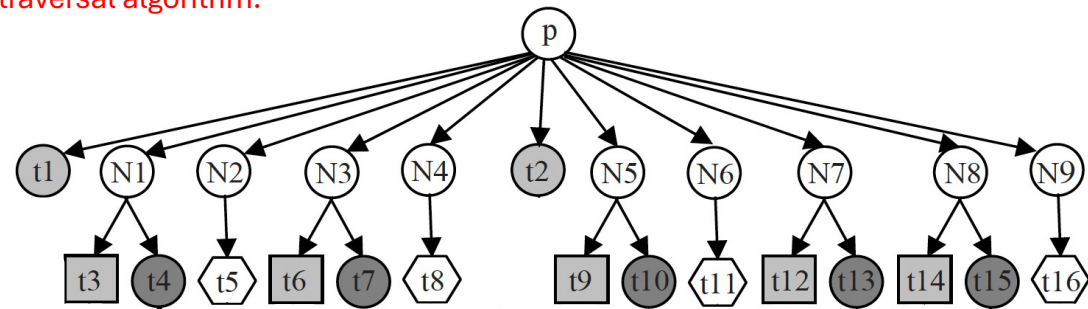


Fig. 9.33. The NET algorithm

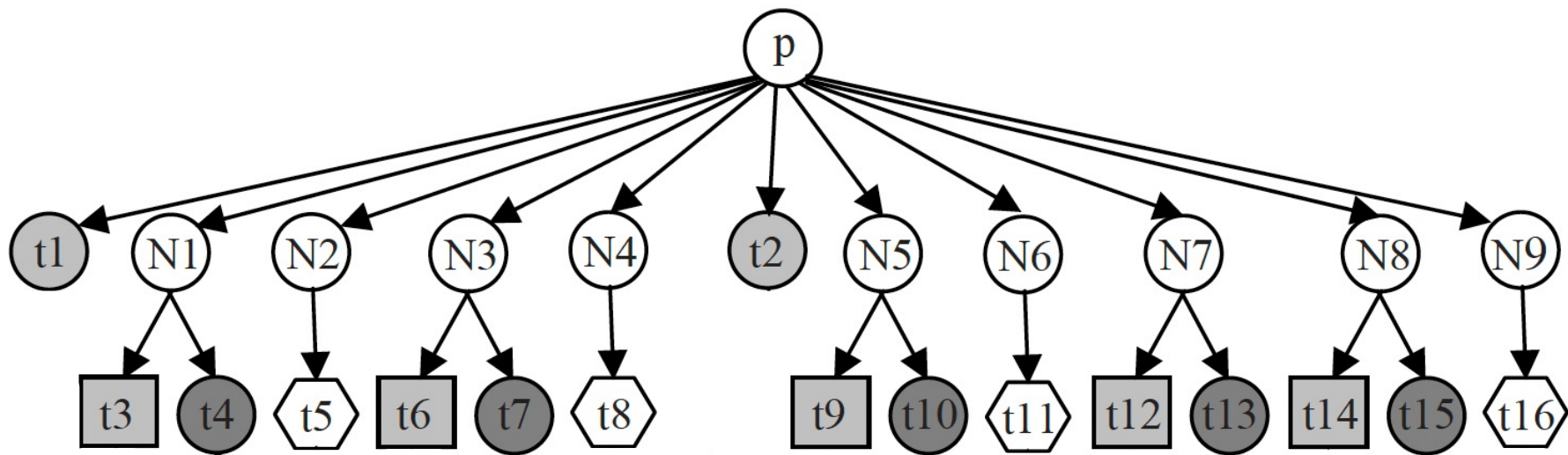
Function Match(*Node*, τ) Performs tree matching on child sub-trees of *Node* and pattern generation

```

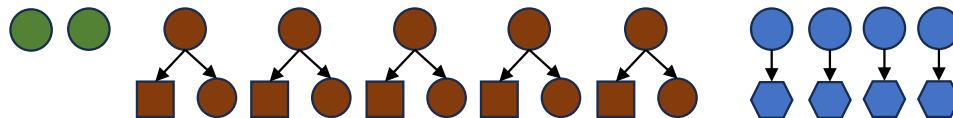
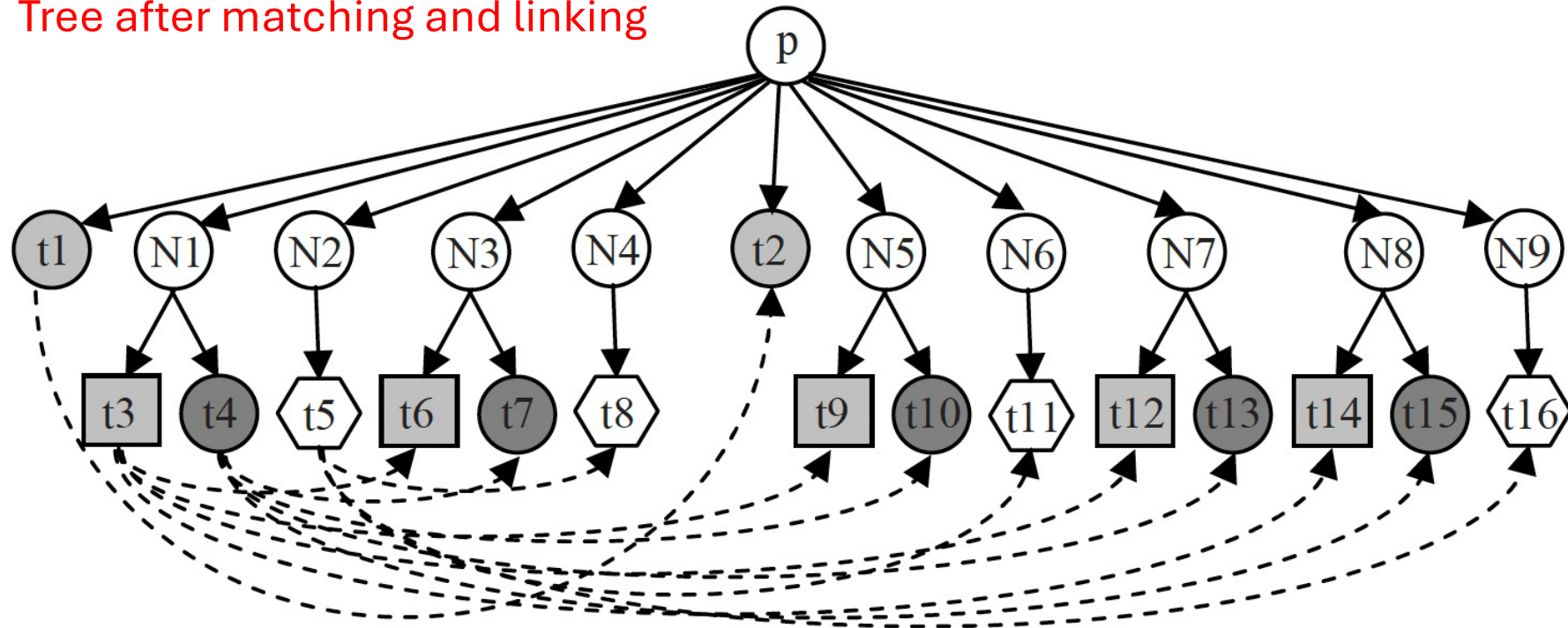
1  Children  $\leftarrow$  Node.Children;
2  while Children  $\neq \emptyset$  do
3    ChildFirst  $\leftarrow$  select and remove the first child from Children;
4    for each ChildR in Children do
5      if TreeMatch(ChildFirst, ChildR)  $> \tau$  then STM algorithm
6        AlignAndLink();
7        Children  $\leftarrow$  Children - {ChildR} Remove ChildR – it has already been matched with ChildFirst
8    endfor
9    if some alignments (or links) have been made with ChildFirst then
10      GenNodePattern(ChildFirst) Partial tree alignment (simplified; pattern can be regular expression)
11  endwhile
12  If consecutive child nodes in Children are aligned then
13    GenRecordPattern(Node) Regular expression representing a record

```

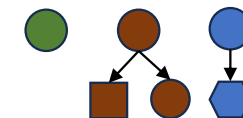
Tree to process



Tree after matching and linking



Matching



Partial Alignment
(GenNodePattern)

Function GenRecordPattern(Node)

```

1  String  $\leftarrow$  Assign a distinctive symbol to each set of matched children of Node;
2  Initialize a data structure for NFA  $N = (Q, \Sigma, \delta, q_0, F)$ , where  $Q$  is the set of
   states,  $\Sigma$  is the symbol set containing all symbols appeared in String,  $\delta$  is the
   transition relation that is a partial function from  $Q \times (\Sigma \cup \{\varepsilon\})$  to  $Q$ , and  $F$  is
   the set of accept states,  $Q \leftarrow \{q_0\}$  ( $q_0$  is the start state),  $\delta \leftarrow \emptyset$  and  $F \leftarrow \emptyset$ ;
3   $q_c \leftarrow q_0$ ; //  $q_c$  is the current state
4  for each symbol  $s$  in String in sequence do
5      if  $\exists$  a transition  $\delta(q_c, s) = q_n$  then
6           $q_c \leftarrow q_n$  // transit to the next state;
7      else if  $\exists \delta(q_i, s) = q_j$ , where  $q_i, q_j \in Q$  then //  $s$  appeared before
8          if  $\exists \delta(q_f, \varepsilon) = q_i$ , where  $\delta(q_i, s) = q_j$  and  $f \geq c$  then
9              TransitTo( $q_c, q_f$ )
10         else TransitTo( $q_c, q_i$ )
11          $q_c \leftarrow q_j$ 
12     else create a new state  $q_{c+1}$  and a transition  $\delta(q_c, s) = q_{c+1}$ ,
           i.e.,  $\delta \leftarrow \delta \cup \{(q_c, s), q_{c+1}\}$ 
13          $Q \leftarrow Q \cup \{q_{c+1}\}$ ;
14          $q_c \leftarrow q_{c+1}$ 
15     if  $s$  is the last symbol in String then
16         Assign the state with the largest subscript the accept state  $q_r$ ,  $F = \{q_r\}$ ;
17         TransitTo( $q_c, q_r$ );
18 endfor
19 generate a regular expression based on the NFA  $N$ ;
20 Substitute all the node patterns into the regular expression.

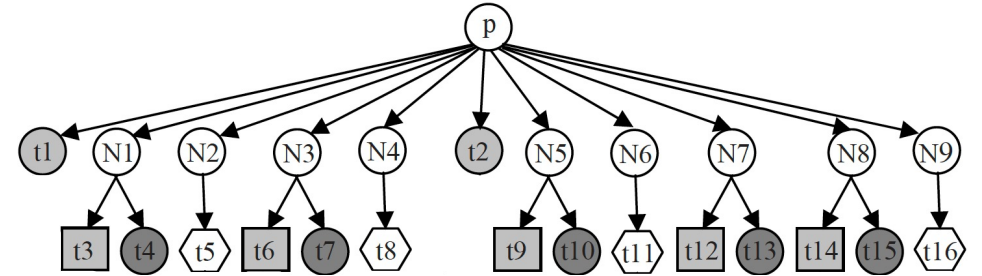
```

Function TransitTo(q_c, q_s)

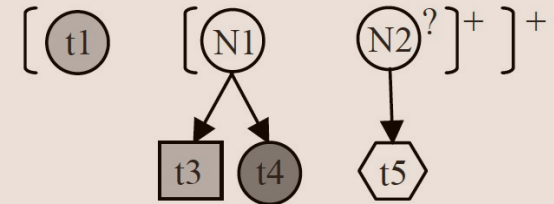
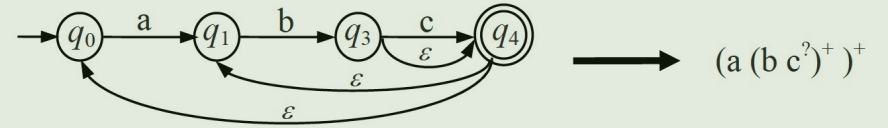
```

1  while  $q_c \neq q_s$  do
2      if  $\exists \delta(q_c, \varepsilon) = q_k$  and  $k > c$  then
3           $q_c \leftarrow q_k$ 
4      else create a transition  $\delta(q_c, \varepsilon) = q_{c+1}$ , i.e.,  $\delta \leftarrow \delta \cup \{(q_c, \varepsilon), q_{c+1}\}$ ;
5           $q_c \leftarrow q_{c+1}$ 
6  endwhile

```



$t1$ $N1$ $N2$ $N3$ $N4$ $t2$ $N5$ $N6$ $N7$ $N8$ $N9$
String: a b c b c a b c b b c



University Curriculum Overview

Study Program T1

N1
Course: t3 Semester: t4
N2
Lecturer: t5

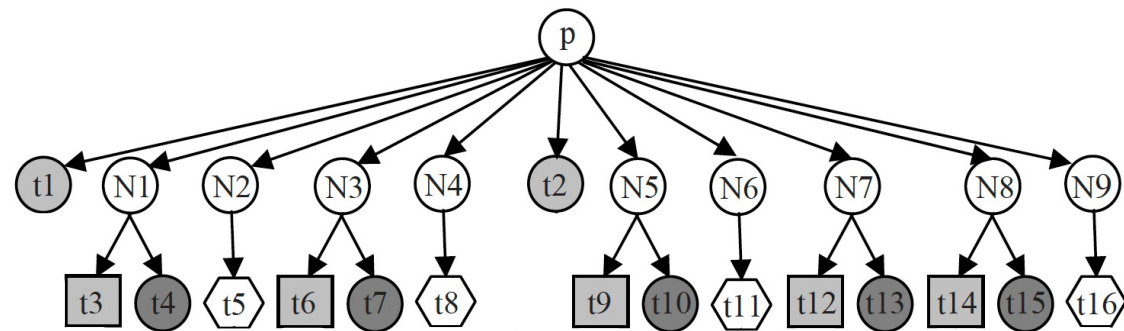
N3
Course: t6 Semester: t7
N4
Lecturer: t8

Study Program T2

N5
Course: t9 Semester: t10
N6
Lecturer: t11

N7
Course: t12 Semester: t13

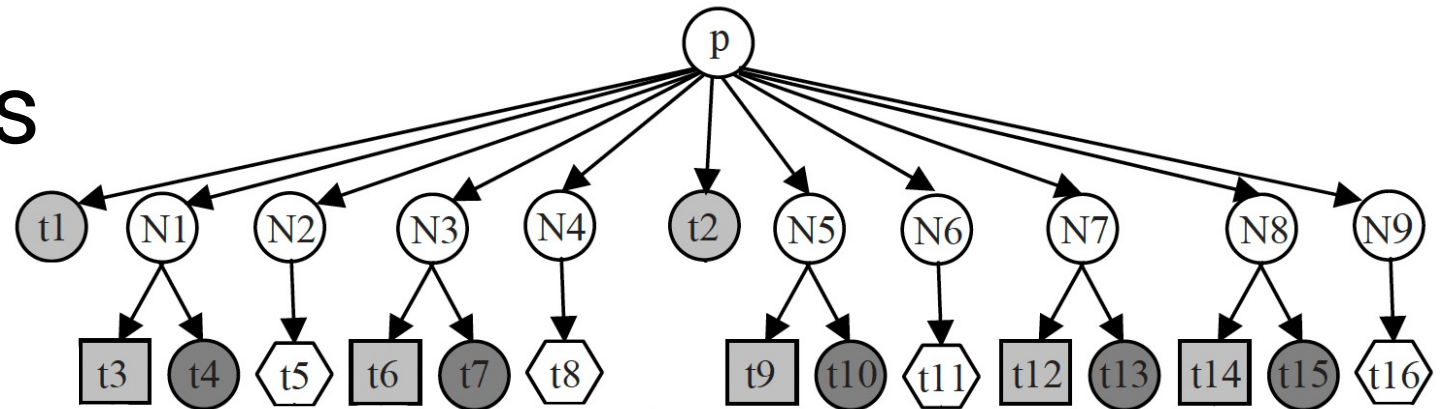
N8
Course: t14 Semester: t15
N9
Lecturer: t16



Study program	Course	Semester	Lecturer
t1	t3	t4	t5
t1	t6	t7	t8
t2	t9	t10	t11
t2	t12	t13	
t2	t14	t15	t16

PutDataInTables

Limitations



- The nodes in the first data record at each level must be complete (t1, N1, N2).
- The first node of every data record at each level must be present (t1, t2; N1, N3, N5, N7, N8).
- Nodes within a single flat data record (no nesting) do not match one another (N1, N3).

Web extraction – multiple pages

- Uncovering patterns for extraction from similar pages
 - List pages
 - Detail pages
- *RoadRunner algorithm* (Crescenzi, Mecca and Merialdo, 2001)
 - Generate a *union-free regular expression* for data extraction from a set of similar pages.
- Union-free regular expression (Brzozowski, 1926)
 - Regular expression without disjunctions

RoadRunner algorithm

■ Approach

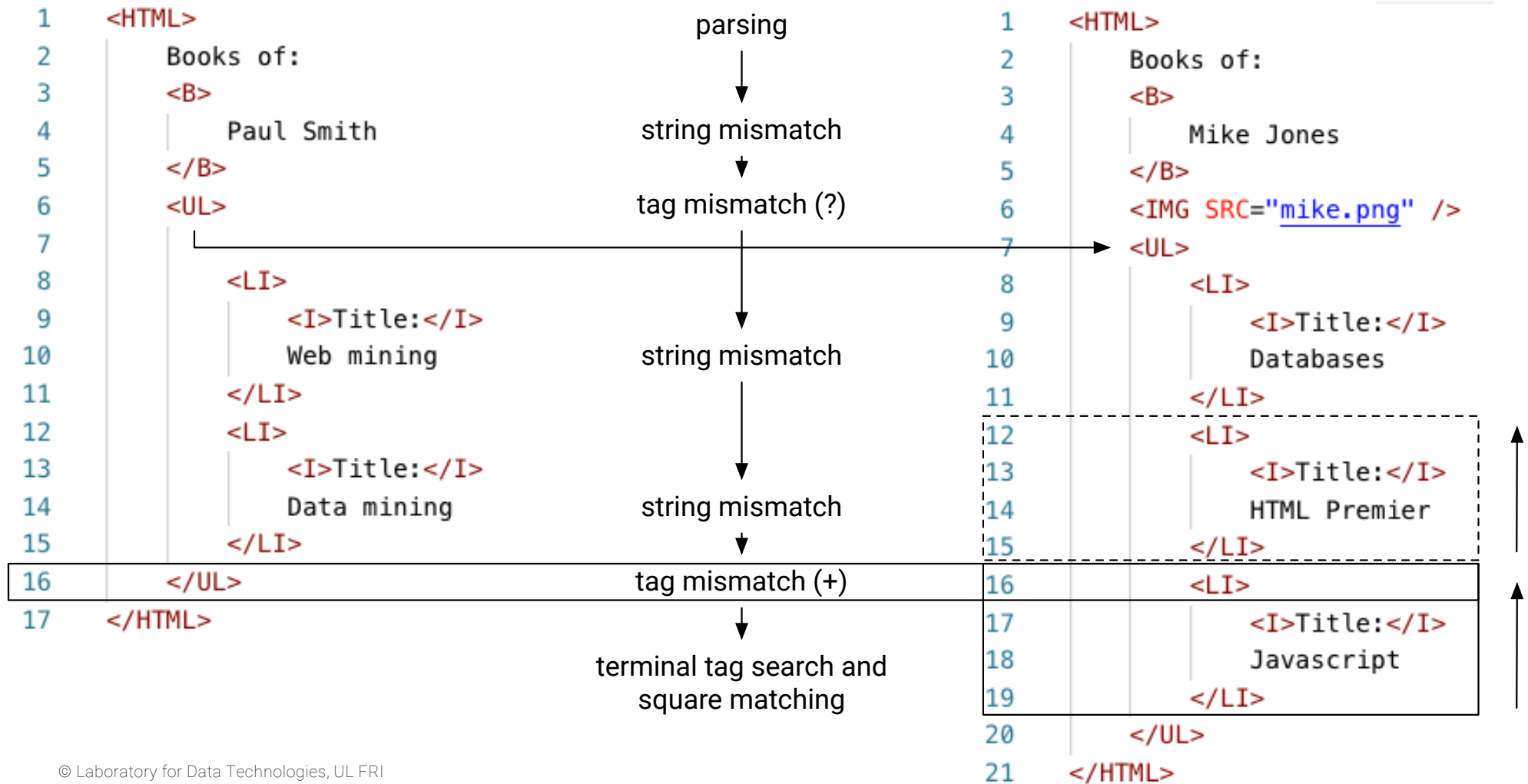
1. Take a random page $p_i \in P$ as a *regular expression wrapper* W
2. For each $p_j \in P \mid i \neq j$ do:
 1. Generalize W by solving mismatches with p_j

■ Types of mismatches

- *Text string mismatches*
 - data fields or items
- *Tag mismatches*
 - optional items
 - iterators (list of items)

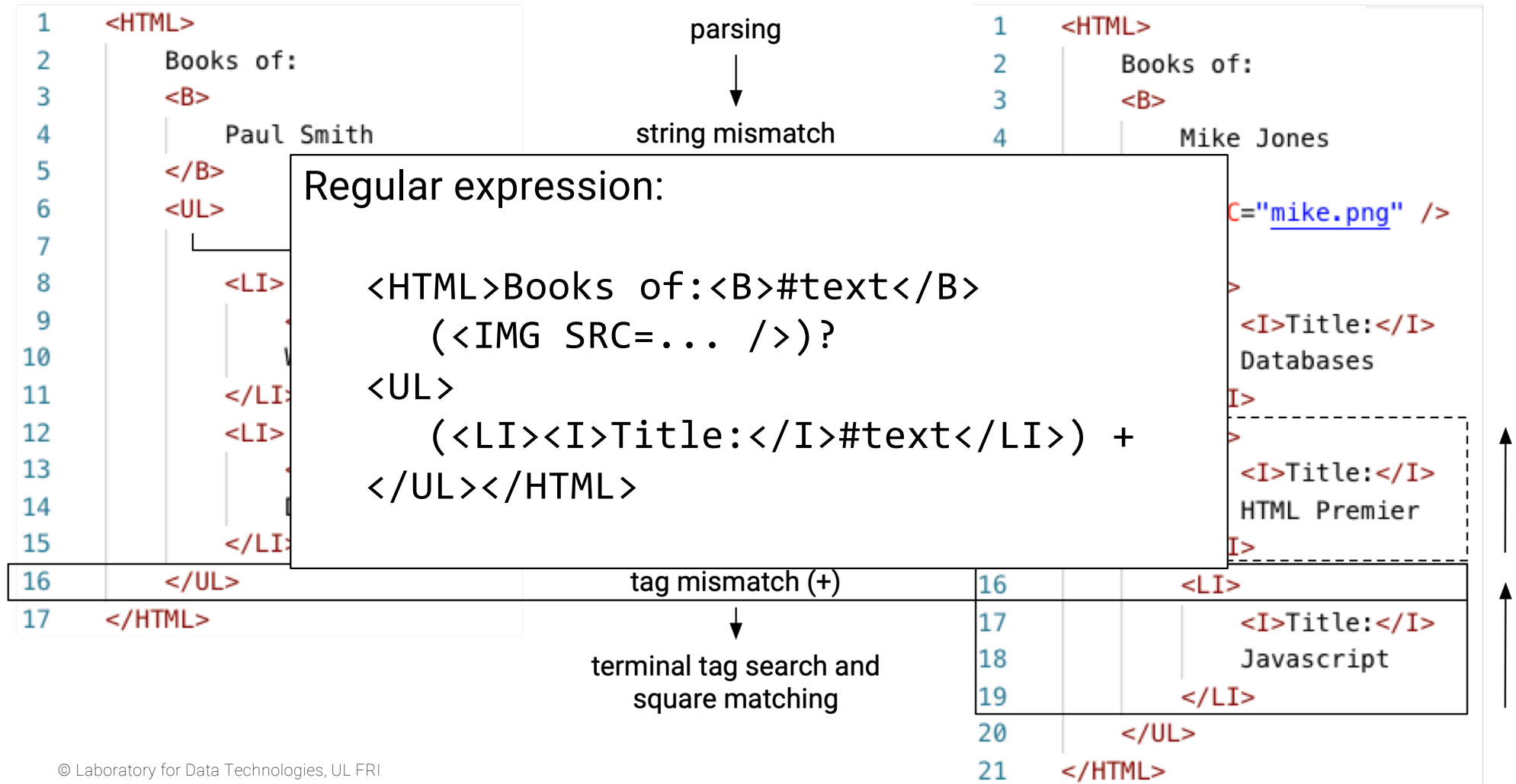
Wrapper page (page 1)

Comparison page (page 2)



Wrapper page (page 1)

Comparison page (page 2)



Automatic Web extraction - Issues

- Extraction based on the encoding template
- Detecting new templates
- Disjunction or optional values
- List of set type or attribute type
- Labeling and integration
- Extraction of unwanted data